Mini project report on

PROPFIND: SEMANTIC PROPERTY SEARCH & RENTAL MANAGEMENT SYSTEM

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE23CS351A – DBMS Project

Submitted by:

KANISHQ SURENDRAN	**PES2UG24CS214**
JISHNU SIVAKUMAR	**PES2UG24CS204**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

PropFind: Semantic Property Search & Rental Management System

is a bonafide work carried out by

KANISHQ SURENDRAN	**PES2UG24CS214**
JISHNU SIVAKUMAR	**PES2UG24CS204**

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CSS351A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature

Dr. Shilpa S

Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled PropFind: Semantic Property Search & Rental Management System has been carried out by us under the guidance of Dr. Shilpa S, Assistant Professor and submitted in partial fulfilment of the course requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester AUG – DEC 2026.

KANISHQ SURENDRAN PES2UG24CS214 <Signature>

JISHNU SIVAKUMAR PES2UG24CS204 <Signature>

ACKNOWLEDGEMENT

We would like to express our gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS351A - DBMS Project.

We take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing us various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from our family and friends.

ABSTRACT

PropFind is a rental-property discovery and management platform built for the DBMS Level 3 mini-project using Option 2: a SQL database combined with a vector database. Tenants describe what they want in plain language, for example "house where I can keep my dog" or "pet friendly 2BHK in Pune under 25k". Every listing is converted into a 384-dimension embedding with the all-MiniLM-L6-v2 sentence model and stored in PostgreSQL using the pgvector extension. A query is embedded the same way and the closest listings are retrieved by cosine distance through an HNSW index. Because vector similarity cannot compare numbers, constraints written in the sentence (budget, bedrooms, city) are parsed and enforced as exact SQL filters, so "homes under rent 20k" returns only homes that really cost under Rs. 20,000.

The same PostgreSQL database holds the relational core: users with roles (owner, tenant, admin), properties, bookings, reviews, payments and a booking audit log. Integrity is enforced inside the database with ENUM types, CHECK, UNIQUE and FOREIGN KEY constraints and seven triggers (double-booking prevention, booking state machine, payment validation and timestamping, audit logging, automatic refund on cancellation and timestamp maintenance). Bookings run as ACID transactions with row-level locking (SELECT ... FOR UPDATE).

The Next.js 14 application provides authentication (bcrypt password hashes, JWT in an httpOnly cookie), role-based access control in middleware and in every API route, owner and tenant dashboards, and a payment flow. The repository contains a 58-listing dataset across 15 cities, labelled SQL demonstrations, a self-checking SQL test script (23 checks, all passing) and parser tests (12 cases).

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	**INTRODUCTION**	**11**
2.	**PROBLEM DEFINITION**	**14**
3.	**ER MODEL**	**16**
4.	**ER TO RELATIONAL MAPPING**	**19**
5.	**DDL STATEMENTS**	**21**
6.	**DML STATEMENTS**	**30**
7.	**QUERIES (SIMPLE QUERY, UPDATE AND DELETE OPERATION, CORRELATED AND NESTED QUERY)**	**33**
8.	**STORED PROCEDURES, FUNCTIONS AND TRIGGERS**	**43**
9.	**FRONT END DEVELOPMENT**	**53**
	REFERENCES / BIBLIOGRAPHY	**69**
	APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS	**70**
	APPENDIX B: DATA DICTIONARY	**71**
	APPENDIX C: EVALUATION RUBRICS - COMPLETION STATUS	**74**

LIST OF TABLES

Table No.	Title	Page No.
1.1	Data in PropFind and why it is stored where it is	**12**
1.2	Technology stack	**12**
1.3	Team members and responsibilities	**13**
2.1	Functional requirements	**14**
2.2	Roles and permissions	**15**
3.1	Entities	**16**
3.2	Relationships and cardinalities	**16**
3.3	How listings are represented as vectors	**17**
4.1	ER to relational mapping steps applied to PropFind	**19**
5.1	Indexes	**25**
5.2	Primary key, foreign key, unique and check constraints in the database	**28**
6.1	Rows in the database	**32**
8.1	Stored functions	**43**
8.2	Triggers	**45**
9.1	Tests performed	**67**

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Entity-relationship diagram of PropFind (7 entities)	**17**
4.1	Relational schema of PropFind with primary keys (underlined) and foreign keys (italic)	**20**
5.1	Executing db/schema.sql (extensions, types, tables, indexes, views, functions and triggers)	**22**
5.2	The seven tables created (\dt)	**22**
5.3	CREATE TABLE users and properties with CHECK constraints	**23**
5.4	Structure of the users table (\d users): primary key, unique and CHECK constraints	**23**
5.5	Vector table property_embeddings, HNSW index and semantic search function	**24**
5.6	Structure of property_embeddings (\d property_embeddings)	**24**
5.7	Structure of bookings (\d bookings): foreign keys, CHECK constraint and triggers	**24**
5.8	CREATE TABLE payments and its trigger	**25**
5.9	Structure of payments (\d payments)	**25**
5.10	ALTER TABLE, CREATE VIEW, DROP VIEW and DROP TABLE (rolled back)	**28**
6.1	Table users (owners, tenants and one admin)	**30**
6.2	Table properties (first 12 of 58 rows) and summary	**31**
6.3	Table property_embeddings: 384-dimension vectors (preview)	**31**
6.4	Table bookings: all statuses	**31**
6.5	Table reviews (first 10 rows)	**31**
6.6	Tables payments and booking_audit (the audit rows were written by a trigger)	**32**
7.1	GROUP BY with COUNT, AVG, MIN, MAX and SUM	**33**

7.2	HAVING and an aggregate over an array column	**34**
7.3	UPDATE with a subquery; the timestamp trigger fires; the transaction is rolled back	**34**
7.4	DELETE with a condition, then ROLLBACK restores the rows	**35**
7.5	Correlated subqueries: comparison with the city average, EXISTS, NOT EXISTS and a scalar subquery	**36**
7.6	Nested subqueries: scalar, IN, derived table and ALL	**37**
7.7	INNER JOIN of four tables, LEFT JOIN and an anti-join	**38**
7.8	RANK() window function, WITH query and UNION	**40**
7.9	Nearest neighbours with the <=> operator and the semantic search function	**41**
7.10	Hybrid query (vector ranking with relational filters) and the HNSW index	**41**
7.11	Full-text search with tsvector and tsquery, and a payments query	**42**
8.1	fn_update_timestamp and fn_check_booking_conflict	**43**
8.2	fn_search_properties_semantic: pgvector cosine search with filters	**44**
8.3	Functions installed in the database (\df fn_*)	**44**
8.4	Views and fn_check_booking_conflict in use	**45**
8.5	Double-booking trigger and booking state machine	**46**
8.6	Audit trigger and automatic refund trigger	**47**
8.7	All triggers installed in the database	**47**
8.8	Triggers in action: audit row written, payment timestamp stamped, ROLLBACK	**48**
8.9	SAVEPOINT, atomicity of a failed transaction and the automatic refund	**49**
8.10	The overlap trigger rejects an approval and the message is shown in the Owner Portal	**50**
8.11	Transactional booking creation with a row lock (lib/services/booking-service.ts)	**51**
8.12	Integrity tests: 22 rejected statements and one accepted control, 23 of 23 passed	**52**
9.1	System architecture	**53**
9.2	Role-based protection of pages (middleware.ts)	**54**

9.3	JWT session helpers and role check (lib/jwt.ts, lib/auth.ts)	**55**
9.4	Login page	**55**
9.5	Registration page (owner or tenant)	**56**
9.6	Home page	**56**
9.7	Search "homes under rent 20k": 23 results, all under Rs. 20,000, with the understood constraint shown	**57**
9.8	Search "house where I can keep my dog": semantic ranking puts pet-friendly homes first	**57**
9.9	Search "pet friendly 2BHK in Pune under 25k": bedrooms, city and budget parsed into SQL filters	**58**
9.10	Extracting budget, bedrooms and city from the sentence (lib/query-parser.ts)	**59**
9.11	Hybrid search service (lib/services/property-service.ts)	**59**
9.12	Generating a 384-dimension embedding (lib/embeddings.ts)	**60**
9.13	Listing details	**60**
9.14	Booking request form (runs as a PostgreSQL transaction)	**61**
9.15	Tenant dashboard: own bookings only, Pay, Cancel and status history from the audit table	**62**
9.16	Listing form (the embedding is created in the same transaction)	**63**
9.17	Owner Portal: four clickable cards (from view_owner_analytics) and booking tabs	**64**
9.18	My Properties	**65**
9.19	Pending requests	**66**
9.20	Confirmed bookings	**66**
9.21	Confirmed revenue: collected and outstanding, per property	**67**

1. INTRODUCTION

1.1 OVERVIEW

PropFind is a rental property discovery and management system. A tenant types what they are looking for in everyday language ("house where I can keep my dog", "pet friendly 2BHK in Pune under 25k") and the system returns the listings that match the meaning of the request, not just the words. Owners publish and manage listings, tenants request, pay for and cancel bookings, and every important rule (no overlapping bookings, valid status changes, valid payments) is enforced inside the database itself.

The project follows Option 2 of the course brief: a SQL database (PostgreSQL) together with a vector database (the pgvector extension of the same PostgreSQL server). A Next.js web application provides the interface and talks to the database only through an API layer.

1.2 MOTIVATION

Checkbox and keyword filters fail when a person only has a vague need, and ordinary booking code fails when two requests for the same dates arrive together. These are exactly the places where database design matters: the first needs semantic retrieval, the second needs transactions, locking and triggers. The project was chosen to connect those database concepts to a realistic application and to go beyond simple create, read, update and delete operations.

1.3 OBJECTIVES

- Design a normalised relational schema with strong integrity rules and document it (ER diagram, relational mapping, data dictionary).
- Add semantic search with embeddings stored and queried inside PostgreSQL and combine it with exact relational filters.
- Prevent double booking and invalid state changes at the database level, independent of application code.
- Provide a working Next.js interface with authentication and at least two roles.
- Demonstrate DDL, DML, joins, grouping, subqueries, views, functions, triggers, indexes, transactions and vector search with reproducible SQL files and tests.

1.4 SCOPE

- **In scope:** registration and login (owner, tenant), listing creation with automatic embedding, natural-language and filtered search, booking request, approval, rejection, cancellation, payment recording, audit history, owner dashboards.
- **Out of scope:** a real payment gateway (a payment is recorded, no money moves), a dedicated admin screen, image upload (images are URLs), e-mail notifications.

1.5 DATABASE PARADIGM AND JUSTIFICATION

The brief allows SQL plus either NoSQL or a vector database. PropFind uses PostgreSQL with pgvector. Listings are structured data (price, city, bedrooms, status) and text-heavy data (descriptions, amenities), so one engine can filter exactly with SQL and rank by meaning with vectors, in the same transaction and with the same foreign keys. A separate NoSQL store such as MongoDB would add a second system without helping the semantic-search requirement.

Table 1.1: Data in PropFind and why it is stored where it is

Data	Stored as	Reason
Users, properties, bookings, reviews, payments	Relational tables in PostgreSQL	Strict structure, relationships, constraints and transactions are required
Booking history	Table booking_audit written by a trigger	Append-only log that must not depend on application code
Listing meaning (embedding)	Column vector(384) in property_embeddings (pgvector)	Similarity search by cosine distance with an HNSW index
Amenities	Array column text[] in properties	Small multivalued attribute that is always read with its property
Sessions	Signed JWT in an httpOnly cookie	Stateless authentication, nothing extra stored in the database

1.6 TECHNOLOGY STACK

Table 1.2: Technology stack

Layer	Technology	Role in PropFind
Frontend	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS	Pages, forms, dashboards, search interface
Backend / API	Next.js Route Handlers on Node.js 24	REST endpoints, validation, role checks
SQL database	PostgreSQL 16.15	Tables, constraints, views, functions, triggers, transactions
Vector database	pgvector 0.8.7 (HNSW index, cosine distance)	Embedding storage and similarity search inside PostgreSQL
Embedding model	all-MiniLM-L6-v2 through @huggingface/transformers (ONNX), 384 dimensions	Turns listings and queries into vectors; runs locally without an API key
Database driver	node-postgres (pg)	Parameterised queries, connection pool, transactions
Authentication	bcryptjs, jose (JWT HS256), httpOnly cookie	Password hashing, sessions, role-based access
Infrastructure	Docker Compose (image pgvector/pgvector:pg16)	Runs the database locally; a hosted option (Supabase, Neon) is documented
Version control	Git and GitHub with feature branches and pull requests	Individual contributions are visible in the history

1.7 TEAM AND CONTRIBUTIONS

Table 1.3: Team members and responsibilities

Name	SRN	Responsibility
KANISHQ SURENDRAN	PES2UG24CS214	Authentication and roles, booking life cycle and concurrency control, payments and audit triggers, real embeddings, number-aware search, dataset, owner and tenant dashboards, SQL demonstrations and tests, documentation (55 commits, 3,673 lines added)
JISHNU SIVAKUMAR	PES2UG24CS204	Database schema and constraints, pgvector storage and HNSW index, semantic search function, search page, property cards and listing form, initial seed pipeline (2 commits, 4,120 lines added)

Line and commit counts were measured with git log on the main branch, excluding merge commits and package-lock.json. The work was merged through pull requests from separate feature branches.

2. PROBLEM DEFINITION

2.1 PROBLEM STATEMENT

Design and implement a rental platform in which tenants can discover homes by describing their needs in natural language, owners can list and manage properties, and the complete booking life cycle (request, approval, payment, cancellation, audit) is consistent even when several users act at the same time.

2.2 LIMITATIONS OF EXISTING APPROACHES

- Keyword search and rigid filters cannot answer "somewhere quiet where I can keep a pet"; a listing that says "pet friendly" is missed if the user types "dog".
- Vector similarity alone is poor with numbers: the sentences "under 20k" and "under 50k" produce almost identical embeddings, so budget cannot be left to the vector part.
- Application-only checks for double booking are unsafe: two simultaneous requests can both pass the check before either is saved (a race condition).
- Rules kept only in application code can be bypassed by another client, a script or a manual edit of the data.

2.3 PROPOSED SOLUTION

- Store a 384-dimension embedding of every listing in PostgreSQL (pgvector) and rank by cosine distance through an HNSW index.
- Parse budget, bedrooms and city from the sentence and apply them as exact SQL conditions; embed only the descriptive remainder.
- Lock the property row (SELECT ... FOR UPDATE) inside a transaction, check conflicts with a stored function, and back everything up with triggers that reject overlapping approvals and invalid status changes.
- Enforce validity with ENUM types and CHECK, UNIQUE and FOREIGN KEY constraints so the database rejects bad data whatever the client.

2.4 FUNCTIONAL REQUIREMENTS

Table 2.1: Functional requirements

ID	Requirement
FR-1	A visitor can register as an owner or a tenant and log in; passwords are stored only as bcrypt hashes.
FR-2	An owner can publish a listing; its embedding is created and stored in the same transaction.
FR-3	Anyone can search listings in natural language and with filters (city, bedrooms, maximum budget).
FR-4	The system extracts budget, bedrooms and city from the sentence and shows what it understood.
FR-5	A tenant can request a booking for a date range; invalid or conflicting requests are rejected.
FR-6	An owner can approve or reject requests for his own properties only.
FR-7	A tenant can cancel a pending or approved booking.

ID	Requirement
FR-8	A tenant can pay once for an approved booking; cancelling a paid booking refunds the payment automatically.
FR-9	Every booking status change is logged with time and user.
FR-10	The owner dashboard shows properties, pending requests, confirmed bookings and revenue (collected and outstanding).
FR-11	Pages and API routes are protected by role; a tenant cannot use owner functions and vice versa.

2.5 DATA INTEGRITY REQUIREMENTS

- No two APPROVED bookings of one property may overlap in time (trigger trg_validate_booking_overlap).
- Booking status may change only along PENDING to APPROVED, REJECTED or CANCELLED and APPROVED to CANCELLED (trigger trg_validate_booking_transition).
- A payment is accepted only for an APPROVED booking, for the exact booking total, once (trigger and partial unique index).
- Prices, areas, ratings, dates, e-mail and phone formats are validated by CHECK constraints.

2.6 USER ROLES

Table 2.2: Roles and permissions

Role	Created by	Permissions
TENANT	Self-registration	Search, request bookings, cancel own bookings, pay for own approved bookings, view own history
OWNER	Self-registration	Everything a visitor can do, publish listings, view own properties, approve or reject requests for own properties, view revenue
ADMIN	Seeded only (not self-registered)	Passes owner and tenant route checks; a dedicated admin screen is not implemented

3. ER MODEL

3.1 ENTITIES AND ATTRIBUTES

Table 3.1: Entities

Entity	Attributes (key attributes in bold)	Notes
User	**id**, full_name, email (unique), password_hash, role, phone, avatar_url	Role is an ENUM: OWNER, TENANT, ADMIN
Property	**id**, title, description, property_type, price_per_month, security_deposit, bedrooms, bathrooms, area_sqft, address, city, state, pincode, status, amenities, image_url	Status ENUM: AVAILABLE, RENTED, MAINTENANCE; amenities is multivalued
Property_Embedding	**property_id**, embedding (384 numbers), model_name, indexed_content	One row per property; the vector part of the design
Booking	**id**, start_date, end_date, total_amount, status, special_requests	Status ENUM: PENDING, APPROVED, REJECTED, CANCELLED
Review	**id**, rating (1 to 5), comment	One review per tenant per property
Payment	**id**, amount, method, status, transaction_ref (unique), paid_at	Method ENUM: UPI, CARD, NET_BANKING
Booking_Audit	**id**, booking_id, old_status, new_status, changed_by, changed_at	Written only by a trigger

3.2 RELATIONSHIPS

Table 3.2: Relationships and cardinalities

Relationship	Entities	Cardinality	Implemented as
owns	User (owner) - Property	1 : N	properties.owner_id (FK)
has vector	Property - Property_Embedding	1 : 1	property_embeddings.property_id is both PK and FK
requests	User (tenant) - Booking	1 : N	bookings.tenant_id (FK)
receives	Property - Booking	1 : N	bookings.property_id (FK)
writes	User (tenant) - Review	1 : N	reviews.tenant_id (FK)
is rated by	Property - Review	1 : N	reviews.property_id (FK)
is paid by	Booking - Payment	1 : N (at most one PAID)	payments.booking_id (FK) plus partial unique index
is logged in	Booking - Booking_Audit	1 : N	booking_audit.booking_id (no FK, by design)

3.3 ER DIAGRAM

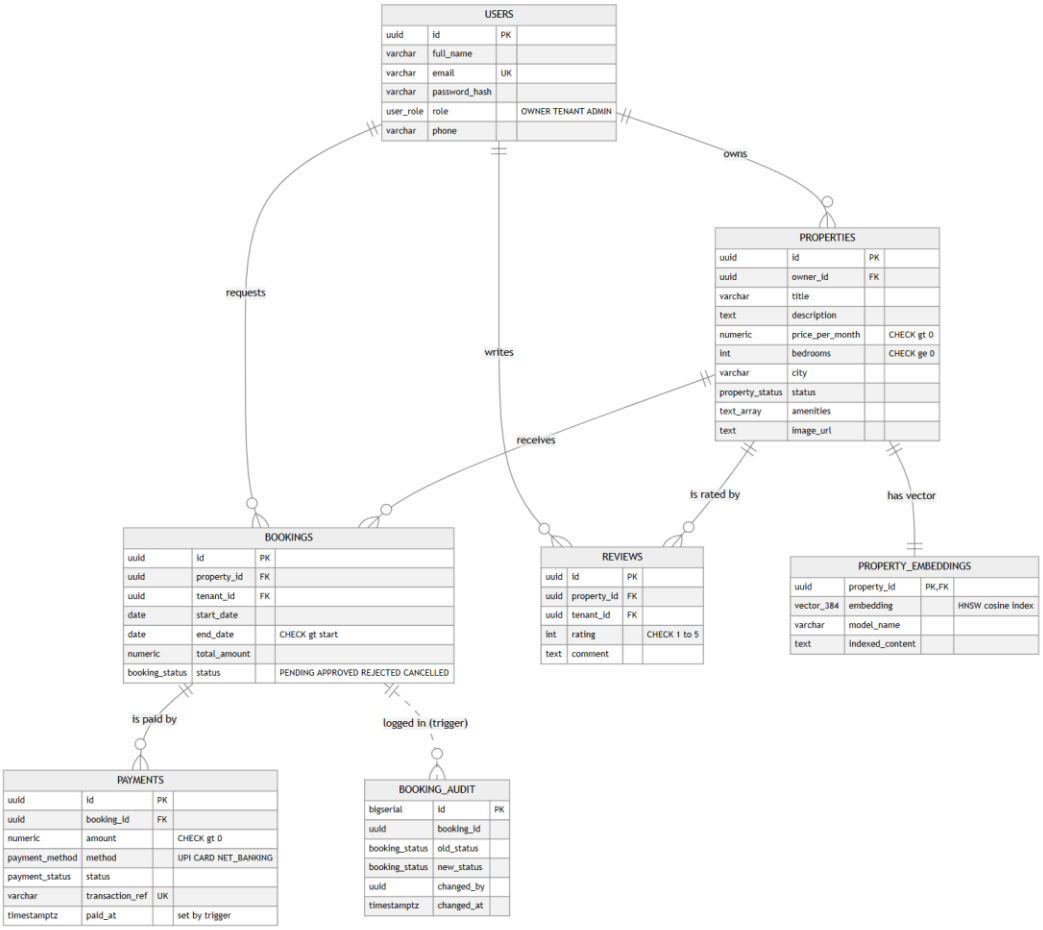


Fig. 3.1: Entity-relationship diagram of PropFind (7 entities)

3.4 BUSINESS RULES CAPTURED BY THE MODEL

- A review is allowed once per tenant per property (UNIQUE on property_id and tenant_id).
- A property with bookings cannot be deleted (ON DELETE RESTRICT); deleting a property removes its embedding and reviews (ON DELETE CASCADE).
- The total amount of a booking is price_per_month multiplied by the number of 30-day months (minimum one).
- A booking end date must be later than its start date.

3.5 VECTOR DATA MODEL

Table 3.3: How listings are represented as vectors

Question	Answer
What is converted to an embedding?	For each listing: title, amenities and description joined into one text (amenities are placed right after the title so short facts such as "Pet Friendly" are not diluted). Each search sentence is embedded too.

Question	Answer
Which model?	all-MiniLM-L6-v2 (sentence-transformers) through @huggingface/transformers: mean pooling and L2 normalisation, 384 dimensions, runs locally.
How are vectors stored?	Table property_embeddings: embedding vector(384), model_name and the exact indexed_content text; one row per property.
Similarity metric	Cosine distance with the <=> operator; similarity = 1 - distance.
Index	HNSW (m = 16, ef_construction = 64) with vector_cosine_ops.
How are results retrieved?	Function fn_search_properties_semantic orders by cosine distance after relational filters (city, price, bedrooms, status).

4. ER TO RELATIONAL MAPPING

4.1 STEPS OF ALGORITHM FOR CHOSEN PROBLEM

The ER diagram was converted to relations with the standard ER-to-relational mapping algorithm (Elmasri and Navathe). Table 4.1 shows each step and how it applies to PropFind.

Table 4.1: ER to relational mapping steps applied to PropFind

Step	Rule	Applied to PropFind
1	Map each regular entity type to a relation; choose a primary key	users, properties, bookings, reviews, payments become relations with a UUID primary key (generated by uuid_generate_v4())
2	Map weak entity types	None. booking_audit is a log and keeps its own surrogate key
3	Map 1:1 relationships	Property - Property_Embedding: property_embeddings.property_id is the primary key and also a foreign key to properties
4	Map 1:N relationships with a foreign key on the N side	owns: properties.owner_id; requests: bookings.tenant_id; receives: bookings.property_id; is paid by: payments.booking_id
5	Map M:N relationships with a relationship relation	User - Property through reviews (tenant, property, rating, comment) with UNIQUE(property_id, tenant_id); User - Property through bookings (dates, amount, status)
6	Map multivalued attributes	amenities is stored as a PostgreSQL text[] array column; a separate table was not needed because amenities are only read with their property
7	Map n-ary relationships	None
8	Map categorical attributes	role, status, method, payment status become ENUM types so invalid values are impossible

Normalisation check

Every relation is in third normal form. All non-key attributes depend only on the primary key: for example properties holds owner_id (a foreign key) rather than the owner name, bookings holds tenant_id and property_id rather than copies of their details, and payment details live in payments instead of bookings. The two deliberate redundancies are total_amount in bookings (a snapshot of the price at booking time, which must not change if the owner later changes the rent) and indexed_content in property_embeddings (the exact text that was embedded, kept for traceability).

4.2 COMPLETE DIAGRAM OF RELATIONAL MAPPING

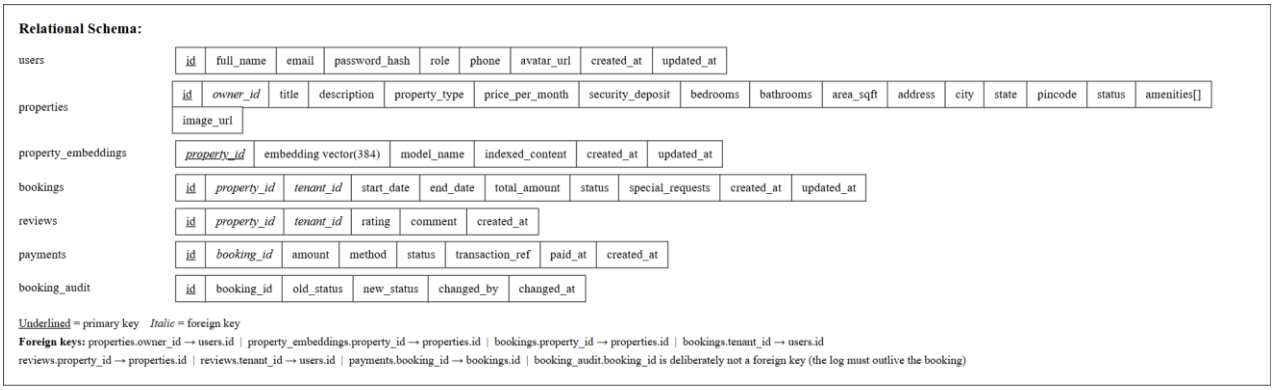


Fig. 4.1: Relational schema of PropFind with primary keys (underlined) and foreign keys (italic)

The resulting database has seven relations, eight foreign-key relationships (the audit log is intentionally unlinked), the constraints listed in Section 5.6 and the indexes listed in Section 5.3.

5. DDL STATEMENTS

STATEMENTS WITH SCREEN SHOTS OF THE TABLE CREATION

5.1 EXTENSIONS AND ENUMERATED TYPES

```
-- 1. Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 3. Custom Enumerations / Domain Types
CREATE TYPE user_role AS ENUM ('OWNER', 'TENANT', 'ADMIN');
CREATE TYPE property_status AS ENUM ('AVAILABLE', 'RENTED', 'MAINTENANCE');
CREATE TYPE booking_status AS ENUM ('PENDING', 'APPROVED', 'REJECTED', 'CANCELLED');
CREATE TYPE payment_method AS ENUM ('UPI', 'CARD', 'NET_BANKING');
CREATE TYPE payment_status AS ENUM ('PAID', 'FAILED', 'REFUNDED');
```

5.2 TABLE CREATION

The complete script is db/schema.sql. It is written to be re-runnable: it drops existing objects first and then creates everything. Figure 5.1 shows the script being executed in psql against a new database and Figure 5.2 lists the seven tables that were created.

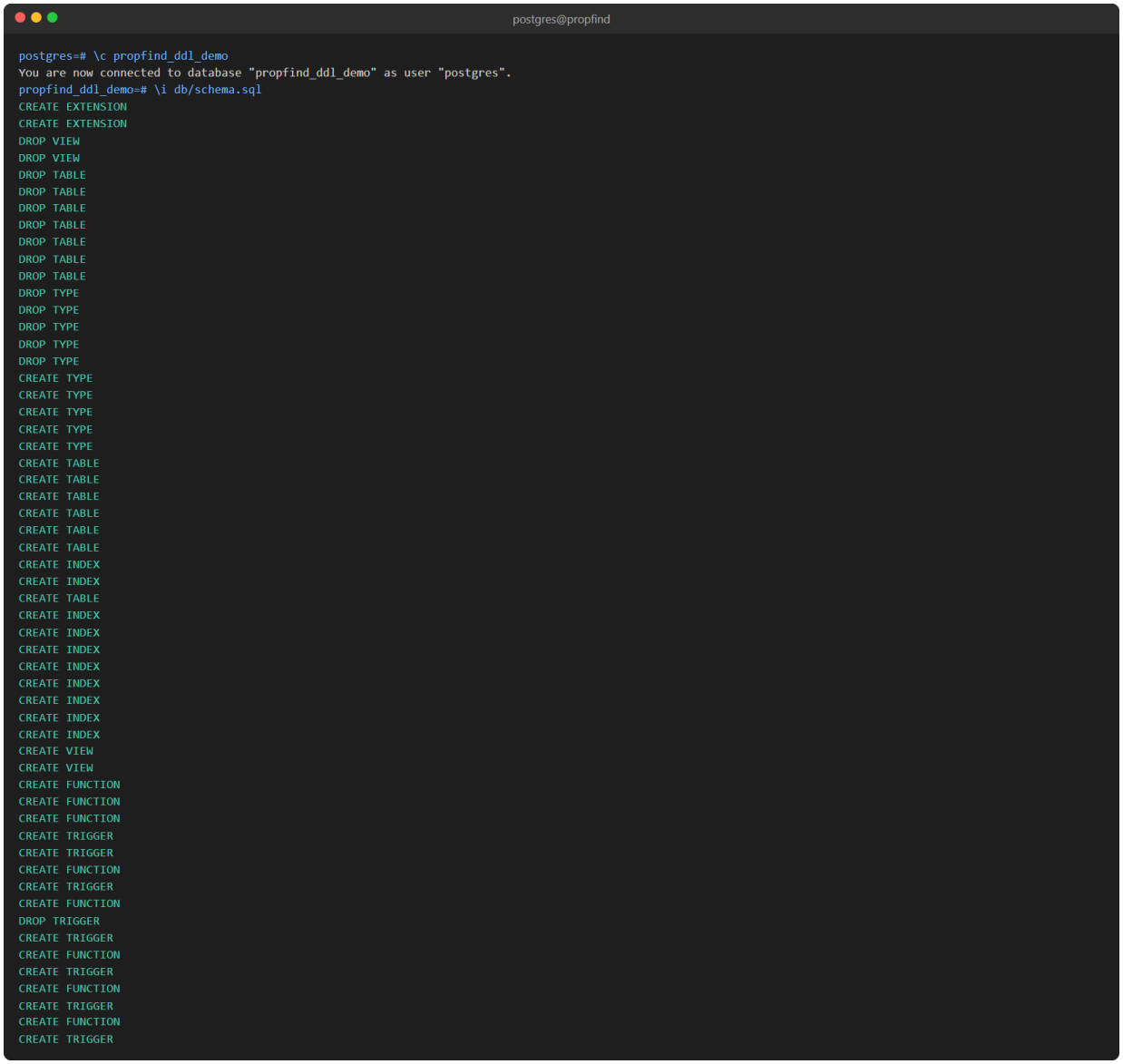


Fig. 5.1: Executing db/schema.sql (extensions, types, tables, indexes, views, functions and triggers)

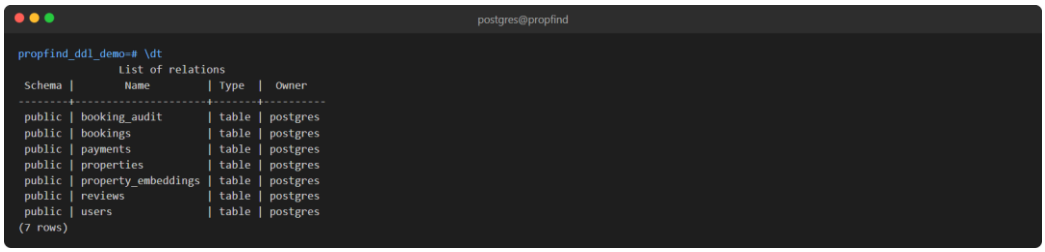


Fig. 5.2: The seven tables created (\dt)

db/schema.sql (users and properties tables with CHECK constraints)

```sql
-- 4. Users Table
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    full_name VARCHAR(120) NOT NULL,
    email VARCHAR(160) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role user_role NOT NULL DEFAULT 'TENANT',
    phone VARCHAR(20),
    avatar_url TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT users_email_format CHECK (email ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
    CONSTRAINT users_phone_format CHECK (phone IS NULL OR phone ~ '^\+?[0-9 ]{10,20}$')
);

-- 5. Properties Table (Relational Catalog)
CREATE TABLE properties (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    owner_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(200) NOT NULL,
    description TEXT NOT NULL,
    property_type VARCHAR(50) NOT NULL DEFAULT 'Apartment',
    price_per_month NUMERIC(10, 2) NOT NULL CHECK (price_per_month > 0),
    security_deposit NUMERIC(10, 2) NOT NULL DEFAULT 0.00 CHECK (security_deposit >= 0),
    bedrooms INT NOT NULL CHECK (bedrooms >= 0),
    bathrooms INT NOT NULL CHECK (bathrooms >= 0),
    area_sqft INT NOT NULL CHECK (area_sqft > 0),
    address TEXT NOT NULL,
    city VARCHAR(80) NOT NULL,
    state VARCHAR(80) NOT NULL,
    pincode VARCHAR(20) NOT NULL,
    status property_status NOT NULL DEFAULT 'AVAILABLE',
    amenities TEXT[] DEFAULT '{}',
    image_url TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Fig. 5.3: CREATE TABLE users and properties with CHECK constraints

postgres@propfind

```
propfind_db=# \d users
                                Table "public.users"
    Column     |           Type           | Collation | Nullable |       Default
---------------+--------------------------+-----------+----------+---------------------
 id            | uuid                     |           | not null | uuid_generate_v4()
 full_name     | character varying(120)   |           | not null |
 email         | character varying(160)   |           | not null |
 password_hash | character varying(255)   |           | not null |
 role          | user_role                |           | not null | 'TENANT'::user_role
 phone         | character varying(20)    |           |          |
 avatar_url    | text                     |           |          |
 created_at    | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
 updated_at    | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "users_pkey" PRIMARY KEY, btree (id)
    "users_email_key" UNIQUE CONSTRAINT, btree (email)
Check constraints:
    "users_email_format" CHECK (email::text ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'::text)
    "users_phone_format" CHECK (phone IS NULL OR phone::text ~ '^\+?[0-9 ]{10,20}$'::text)
Referenced by:
    TABLE "bookings" CONSTRAINT "bookings_tenant_id_fkey" FOREIGN KEY (tenant_id) REFERENCES users(id) ON DELETE RESTRICT
    TABLE "properties" CONSTRAINT "properties_owner_id_fkey" FOREIGN KEY (owner_id) REFERENCES users(id) ON DELETE CASCADE
    TABLE "reviews" CONSTRAINT "reviews_tenant_id_fkey" FOREIGN KEY (tenant_id) REFERENCES users(id) ON DELETE CASCADE
```

Fig. 5.4: Structure of the users table (\d users): primary key, unique and CHECK constraints

db/schema.sql (vector table, HNSW index, semantic search function)

```sql
-- 6. Property Embeddings Table (Vector Database Paradigm)
-- 384 dimensions corresponds to sentence-transformers/all-MiniLM-L6-v2
CREATE TABLE property_embeddings (
    property_id UUID PRIMARY KEY REFERENCES properties(id) ON DELETE CASCADE,
    embedding vector(384) NOT NULL,
    model_name VARCHAR(100) NOT NULL DEFAULT 'all-MiniLM-L6-v2',
    indexed_content TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- pgvector Hierarchical Navigable Small World (HNSW) cosine index
-- Provides sub-linear search time over high-dimensional vector spaces
CREATE INDEX idx_property_embeddings_hnsw
ON property_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- ------------------------------------------------------------------------
```

Fig. 5.5: Vector table property_embeddings, HNSW index and semantic search function

postgres@propfind

```
propfind_db=# \d property_embeddings
                                Table "public.property_embeddings"
     Column      |           Type           | Collation | Nullable |               Default
-----------------+--------------------------+-----------+----------+-------------------------------------
 property_id     | uuid                     |           | not null |
 embedding       | vector(384)              |           | not null |
 model_name      | character varying(100)   |           | not null | 'all-MiniLM-L6-v2'::character varying
 indexed_content | text                     |           | not null |
 created_at      | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
 updated_at      | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "property_embeddings_pkey" PRIMARY KEY, btree (property_id)
    "idx_property_embeddings_hnsw" hnsw (embedding vector_cosine_ops) WITH (m='16', ef_construction='64')
Foreign-key constraints:
    "property_embeddings_property_id_fkey" FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE CASCADE
```

Fig. 5.6: Structure of property_embeddings (\d property_embeddings)

postgres@propfind

```
propfind_db=# \d bookings
                                  Table "public.bookings"
      Column      |           Type           | Collation | Nullable |          Default
------------------+--------------------------+-----------+----------+---------------------------
 id               | uuid                     |           | not null | uuid_generate_v4()
 property_id      | uuid                     |           | not null |
 tenant_id        | uuid                     |           | not null |
 start_date       | date                     |           | not null |
 end_date         | date                     |           | not null |
 total_amount     | numeric(10,2)            |           | not null |
 status           | booking_status           |           | not null | 'PENDING'::booking_status
 special_requests | text                     |           |          |
 created_at       | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
 updated_at       | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "bookings_pkey" PRIMARY KEY, btree (id)
    "idx_bookings_dates" btree (property_id, start_date, end_date)
    "idx_bookings_tenant" btree (tenant_id)
Check constraints:
    "bookings_total_amount_check" CHECK (total_amount >= 0::numeric)
    "check_booking_dates" CHECK (end_date > start_date)
Foreign-key constraints:
    "bookings_property_id_fkey" FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE RESTRICT
    "bookings_tenant_id_fkey" FOREIGN KEY (tenant_id) REFERENCES users(id) ON DELETE RESTRICT
Referenced by:
    TABLE "payments" CONSTRAINT "payments_booking_id_fkey" FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE RESTRICT
Triggers:
    trg_audit_booking_status AFTER INSERT OR UPDATE OF status ON bookings FOR EACH ROW EXECUTE FUNCTION fn_trg_audit_booking_status()
    trg_bookings_timestamp BEFORE UPDATE ON bookings FOR EACH ROW EXECUTE FUNCTION fn_update_timestamp()
    trg_refund_on_cancel AFTER UPDATE OF status ON bookings FOR EACH ROW EXECUTE FUNCTION fn_trg_refund_on_cancel()
    trg_validate_booking_overlap BEFORE INSERT OR UPDATE ON bookings FOR EACH ROW EXECUTE FUNCTION fn_trg_prevent_double_booking()
    trg_validate_booking_transition BEFORE UPDATE OF status ON bookings FOR EACH ROW EXECUTE FUNCTION fn_trg_validate_booking_transition()
```

Fig. 5.7: Structure of bookings (\d bookings): foreign keys, CHECK constraint and triggers

db/schema.sql (payments table and payment trigger)

```sql
-- 9. Payments Table (Member 2): one PAID payment per approved booking
CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE RESTRICT,
    amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
    method payment_method NOT NULL,
    status payment_status NOT NULL DEFAULT 'PAID',
    transaction_ref VARCHAR(40) NOT NULL UNIQUE
        DEFAULT ('TXN-' || upper(substr(md5(random()::text || clock_timestamp()::text), 1, 12))),
    paid_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT payments_paid_has_timestamp CHECK (status <> 'PAID' OR paid_at IS NOT NULL)
);
-- A booking can only be paid once (REFUNDED / FAILED rows do not block a new attempt)
CREATE UNIQUE INDEX uq_payments_one_paid_per_booking ON payments(booking_id) WHERE status = 'PAID';
CREATE INDEX idx_payments_booking ON payments(booking_id);

-- Trigger 5: Stamp paid_at and validate payments (BEFORE INSERT/UPDATE on payments)
--   * paid_at is set automatically when status becomes PAID
--   * a PAID payment is only allowed for an APPROVED booking and must equal total_amount
CREATE OR REPLACE FUNCTION fn_trg_payment_stamp_and_validate()
RETURNS TRIGGER AS $$
DECLARE
    b_status booking_status;
    b_total NUMERIC(10, 2);
BEGIN
    IF NEW.status = 'PAID' AND (TG_OP = 'INSERT' OR OLD.status IS DISTINCT FROM 'PAID') THEN
        SELECT status, total_amount INTO b_status, b_total FROM bookings WHERE id = NEW.booking_id;
        IF b_status IS DISTINCT FROM 'APPROVED' THEN
            RAISE EXCEPTION 'Payment rejected: booking % is %, only APPROVED bookings can be paid',
                NEW.booking_id, COALESCE(b_status::text, 'missing');
        END IF;
        IF NEW.amount <> b_total THEN
            RAISE EXCEPTION 'Payment rejected: amount % does not match booking total %', NEW.amount, b_total;
        END IF;
        NEW.paid_at := CURRENT_TIMESTAMP;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Fig. 5.8: CREATE TABLE payments and its trigger

postgres@propfind

```
propfind_db=# \d payments
                                                Table "public.payments"
     Column      |           Type           | Collation | Nullable |                                  Default                                   ...
-----------------+--------------------------+-----------+----------+----------------------------------------------------------------------------...
 id              | uuid                     |           | not null | uuid_generate_v4()
 booking_id      | uuid                     |           | not null |
 amount          | numeric(10,2)            |           | not null |
 method          | payment_method           |           | not null |
 status          | payment_status           |           | not null | 'PAID'::payment_status
 transaction_ref | character varying(40)    |           | not null | (('TXN-'::text || upper(substr(md5(random()::text || clock_timestamp...
 paid_at         | timestamp with time zone |           |          |
 created_at      | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "payments_pkey" PRIMARY KEY, btree (id)
    "idx_payments_booking" btree (booking_id)
    "payments_transaction_ref_key" UNIQUE CONSTRAINT, btree (transaction_ref)
    "uq_payments_one_paid_per_booking" UNIQUE, btree (booking_id) WHERE status = 'PAID'::payment_status
Check constraints:
    "payments_amount_check" CHECK (amount > 0::numeric)
    "payments_paid_has_timestamp" CHECK (status <> 'PAID'::payment_status OR paid_at IS NOT NULL)
Foreign-key constraints:
    "payments_booking_id_fkey" FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE RESTRICT
Triggers:
    trg_payment_stamp_and_validate BEFORE INSERT OR UPDATE ON payments FOR EACH ROW EXECUTE FUNCTION fn_trg_payment_stamp_and_validate()
```

Fig. 5.9: Structure of payments (\d payments)

5.3 INDEXES

Table 5.1: Indexes

Index	Table and columns	Purpose
idx_properties_owner_id	properties(owner_id)	List an owner's properties
idx_properties_city	properties(city)	City filter

Index	Table and columns	Purpose
idx_properties_price	properties(price_per_month)	Budget filter and price ordering
idx_properties_status	properties(status)	Only AVAILABLE listings are searched
idx_bookings_dates	bookings(property_id, start_date, end_date)	Date-conflict checks
idx_bookings_tenant	bookings(tenant_id)	Tenant booking list
idx_payments_booking	payments(booking_id)	Payments of a booking
idx_booking_audit_booking	booking_audit(booking_id, changed_at)	Booking history
uq_payments_one_paid_per_booking	payments(booking_id) WHERE status = 'PAID' (partial unique)	A booking can be paid only once
idx_property_embeddings_hnsw	property_embeddings USING hnsw (embedding vector_cosine_ops)	Approximate nearest-neighbour search

5.4 VIEWS

```
-- 1. Property Catalog View (Relational Join + Rating Aggregates)
CREATE OR REPLACE VIEW view_property_catalog AS
SELECT
    p.id,
    p.title,
    p.description,
    p.property_type,
    p.price_per_month,
    p.security_deposit,
    p.bedrooms,
    p.bathrooms,
    p.area_sqft,
    p.address,
    p.city,
    p.state,
    p.pincode,
    p.status,
    p.amenities,
    p.image_url,
    p.created_at,
    u.id AS owner_id,
    u.full_name AS owner_name,
    u.email AS owner_email,
    u.phone AS owner_phone,
    COALESCE(ROUND(AVG(r.rating)::numeric, 1), 0.0) AS avg_rating,
    COUNT(r.id) AS review_count
FROM properties p
JOIN users u ON p.owner_id = u.id
LEFT JOIN reviews r ON p.id = r.property_id
GROUP BY p.id, u.id;

-- 2. Owner Analytics View (Aggregate Functions, GROUP BY, HAVING)
CREATE OR REPLACE VIEW view_owner_analytics AS
SELECT
    u.id AS owner_id,
    u.full_name AS owner_name,
    u.email AS owner_email,
    COUNT(DISTINCT p.id) AS total_properties,
    COUNT(DISTINCT CASE WHEN p.status = 'AVAILABLE' THEN p.id END) AS available_properties,
    COUNT(DISTINCT CASE WHEN p.status = 'RENTED' THEN p.id END) AS rented_properties,
    COALESCE(ROUND(AVG(p.price_per_month)::numeric, 2), 0.00) AS avg_monthly_rent,
    COUNT(DISTINCT b.id) AS total_booking_requests,
    COUNT(DISTINCT CASE WHEN b.status = 'APPROVED' THEN b.id END) AS confirmed_bookings,
    COALESCE(SUM(CASE WHEN b.status = 'APPROVED' THEN b.total_amount ELSE 0 END), 0.00) AS
total_confirmed_revenue
FROM users u
LEFT JOIN properties p ON u.id = p.owner_id
LEFT JOIN bookings b ON p.id = b.property_id
```

```
WHERE u.role = 'OWNER'
GROUP BY u.id
HAVING COUNT(p.id) >= 0;

-- ------------------------------------------------------------------------
```

5.5 ALTER AND DROP STATEMENTS

ALTER and DROP are demonstrated inside a transaction that is rolled back, so the real schema is not changed. A new column and a CHECK constraint are added, the column is renamed, a bad value is rejected by the new constraint, and the column is dropped again; a view and a table are created and dropped.

```
-- L1. ALTER TABLE: add a column, add a CHECK constraint, rename, set NOT NULL, drop
BEGIN;
ALTER TABLE properties ADD COLUMN furnishing VARCHAR(20) DEFAULT 'UNFURNISHED';
ALTER TABLE properties
    ADD CONSTRAINT properties_furnishing_check
    CHECK (furnishing IN ('UNFURNISHED', 'SEMI_FURNISHED', 'FULLY_FURNISHED'));
UPDATE properties SET furnishing = 'FULLY_FURNISHED' WHERE 'Fully Furnished' = ANY(amenities);
UPDATE properties SET furnishing = 'SEMI_FURNISHED' WHERE 'Semi-Furnished' = ANY(amenities);
SELECT furnishing, COUNT(*) AS properties FROM properties GROUP BY furnishing ORDER BY properties
DESC;
ALTER TABLE properties RENAME COLUMN furnishing TO furnishing_level;
ALTER TABLE properties ALTER COLUMN furnishing_level SET NOT NULL;
SAVEPOINT before_bad_value;
\set ON_ERROR_STOP off
UPDATE properties SET furnishing_level = 'PARTLY' WHERE id = (SELECT id FROM properties LIMIT 1);  --
violates the new CHECK, error expected
\set ON_ERROR_STOP on
ROLLBACK TO SAVEPOINT before bad value;
ALTER TABLE properties DROP COLUMN furnishing_level;   -- also drops its CHECK constraint
ROLLBACK;
SELECT COUNT(*) AS furnishing_columns_left
FROM information_schema.columns WHERE table_name = 'properties' AND column_name LIKE 'furnishing%';

-- L2. CREATE VIEW / DROP VIEW / DROP TABLE (rolled back)
BEGIN;
CREATE VIEW view_budget_listings AS
    SELECT id, title, city, price_per_month FROM properties WHERE price_per_month < 20000;
SELECT COUNT(*) AS budget_listings, MIN(price_per_month) AS cheapest, MAX(price_per_month) AS dearest
FROM view_budget_listings;
DROP VIEW view_budget_listings;
CREATE TABLE demo_scratch (id INT PRIMARY KEY, note TEXT);
INSERT INTO demo_scratch VALUES (1, 'temporary');
DROP TABLE demo_scratch;
ROLLBACK;
SELECT to_regclass('view_budget_listings') AS view_exists, to_regclass('demo_scratch') AS
table_exists;
```

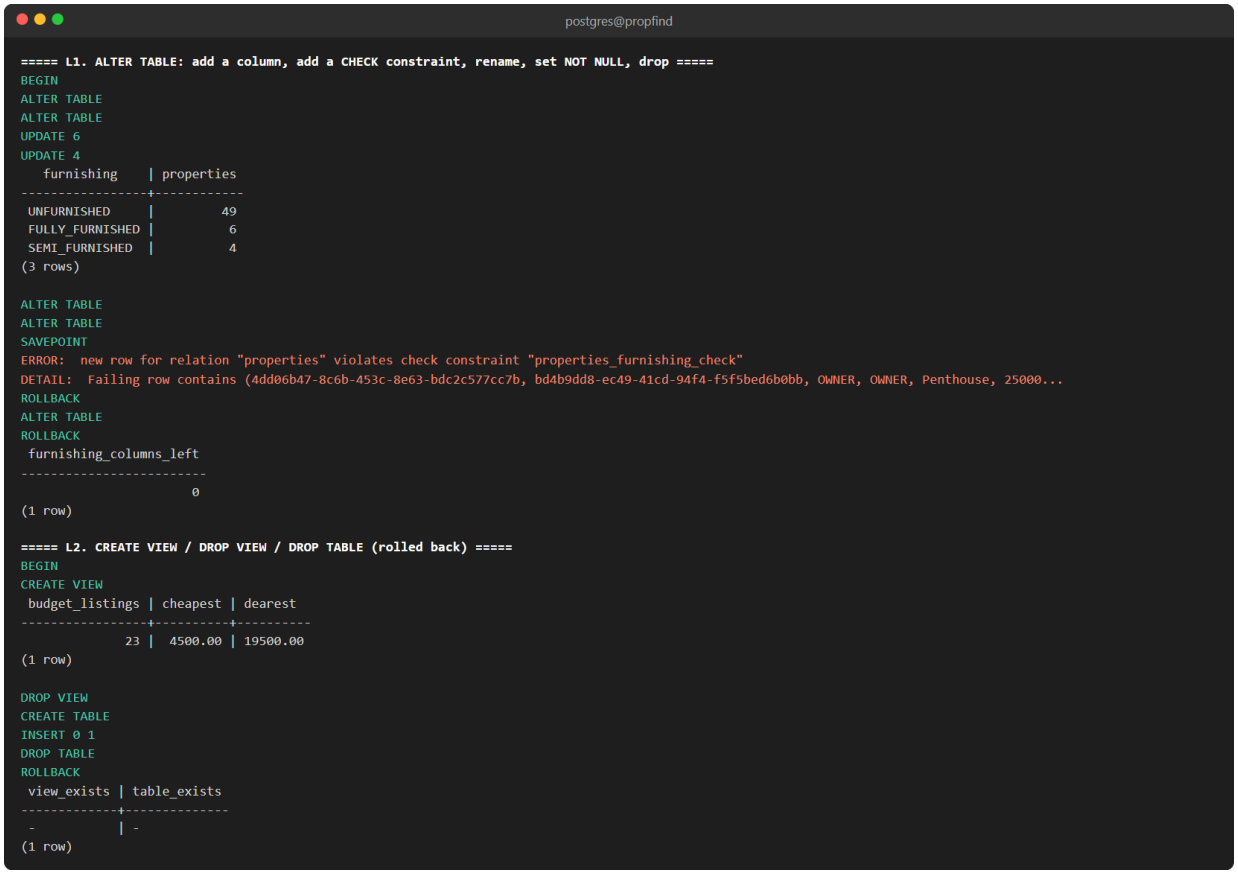


Fig. 5.10: ALTER TABLE, CREATE VIEW, DROP VIEW and DROP TABLE (rolled back)

5.6 CONSTRAINTS SUMMARY

Table 5.2: Primary key, foreign key, unique and check constraints in the database

Table	Constraint	Type	Definition
users	users_email_format	CHECK	CHECK (((email)::text ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'::text))
users	users_phone_format	CHECK	CHECK (((phone IS NULL) OR ((phone)::text ~ '^\+?[0-9]{10,20}$'::text)))
users	users_pkey	PRIMARY KEY	PRIMARY KEY (id)
users	users_email_key	UNIQUE	UNIQUE (email)
properties	properties_area_sqft_check	CHECK	CHECK ((area_sqft > 0))
properties	properties_bathrooms_check	CHECK	CHECK ((bathrooms >= 0))
properties	properties_bedrooms_check	CHECK	CHECK ((bedrooms >= 0))
properties	properties_price_per_month_check	CHECK	CHECK ((price_per_month > (0)::numeric))
properties	properties_security_deposit_check	CHECK	CHECK ((security_deposit >= (0)::numeric))
properties	properties_owner_id_fkey	FOREIGN KEY	FOREIGN KEY (owner_id) REFERENCES users(id) ON DELETE CASCADE
properties	properties_pkey	PRIMARY KEY	PRIMARY KEY (id)
property_embeddings	property_embeddings_property_id_fkey	FOREIGN KEY	FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE CASCADE
property_embeddings	property_embeddings_pkey	PRIMARY KEY	PRIMARY KEY (property_id)
bookings	bookings_total_amount_check	CHECK	CHECK ((total_amount >= (0)::numeric))
bookings	check_booking_dates	CHECK	CHECK ((end_date > start_date))
bookings	bookings_property_id_fkey	FOREIGN KEY	FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE RESTRICT

Table	Constraint	Type	Definition
bookings	bookings_tenant_id_fkey	FOREIGN KEY	FOREIGN KEY (tenant_id) REFERENCES users(id) ON DELETE RESTRICT
bookings	bookings_pkey	PRIMARY KEY	PRIMARY KEY (id)
reviews	reviews_rating_check	CHECK	CHECK (((rating >= 1) AND (rating <= 5)))
reviews	reviews_property_id_fkey	FOREIGN KEY	FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE CASCADE
reviews	reviews_tenant_id_fkey	FOREIGN KEY	FOREIGN KEY (tenant_id) REFERENCES users(id) ON DELETE CASCADE
reviews	reviews_pkey	PRIMARY KEY	PRIMARY KEY (id)
reviews	unique_tenant_property_review	UNIQUE	UNIQUE (property_id, tenant_id)
payments	payments_amount_check	CHECK	CHECK ((amount > (0)::numeric))
payments	payments_paid_has_timestamp	CHECK	CHECK (((status <> 'PAID'::payment_status) OR (paid_at IS NOT NULL)))
payments	payments_booking_id_fkey	FOREIGN KEY	FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE RESTRICT
payments	payments_pkey	PRIMARY KEY	PRIMARY KEY (id)
payments	payments_transaction_ref_key	UNIQUE	UNIQUE (transaction_ref)
booking_audit	booking_audit_pkey	PRIMARY KEY	PRIMARY KEY (id)

6. DML STATEMENTS

STATEMENTS WITH SCREEN SHOTS OF THE TABLE WITH INSERTED VALUES

6.1 INSERT STATEMENTS

Data is inserted by the seed script (scripts/seed.mjs, data in scripts/seed-data.mjs) and by the application. All values are passed as bind parameters, never concatenated into the SQL text. Bookings are inserted as PENDING and then moved through the real state machine, so the audit and payment triggers run exactly as they do for a real user.

```
-- users (password stored as a bcrypt hash)
INSERT INTO users (full_name, email, password_hash, role, phone)
VALUES ($1, $2, $3, $4::user_role, $5) RETURNING id;

-- properties
INSERT INTO properties (owner_id, title, description, property_type, price_per_month,
    security_deposit, bedrooms, bathrooms, area_sqft, address, city, state, pincode,
    status, amenities, image_url)
VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, 'AVAILABLE', $14, $15) RETURNING
*;

-- vector embedding of a property (384 numbers passed as a vector literal)
INSERT INTO property_embeddings (property_id, embedding, model_name, indexed_content)
VALUES ($1, $2::vector, $3, $4);

-- reviews, bookings, payments
INSERT INTO reviews (property_id, tenant_id, rating, comment) VALUES ($1, $2, $3, $4);
INSERT INTO bookings (property_id, tenant_id, start_date, end_date, total_amount,
special_requests)
VALUES ($1, $2, $3, $4, $5, $6) RETURNING id;
INSERT INTO payments (booking_id, amount, method, status) VALUES ($1, $2, 'UPI', 'PAID');
```

6.2 TABLES WITH INSERTED VALUES

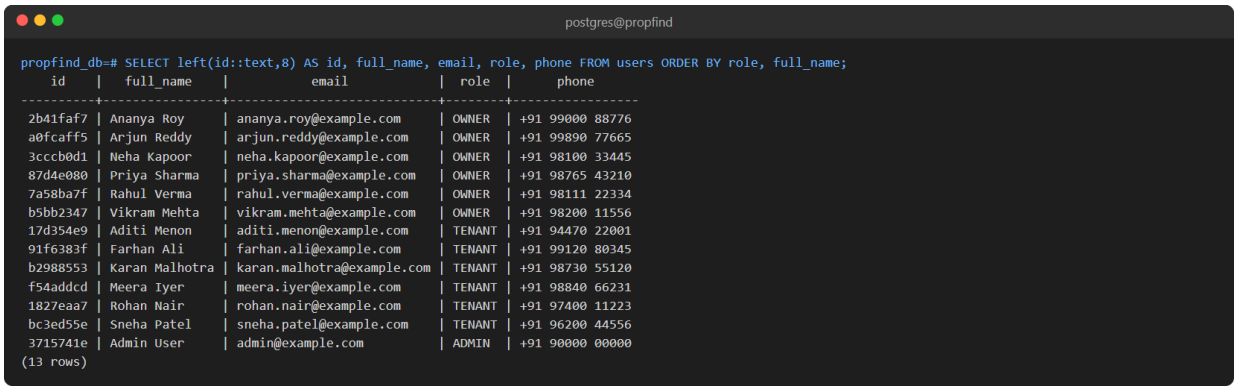


Fig. 6.1: Table users (owners, tenants and one admin)

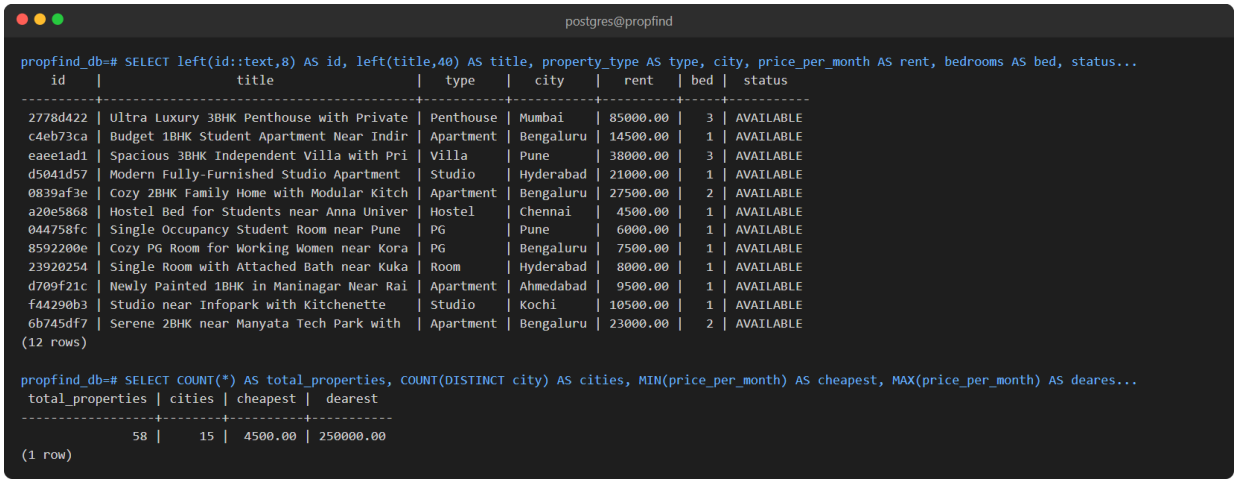


Fig. 6.2: Table properties (first 12 of 58 rows) and summary

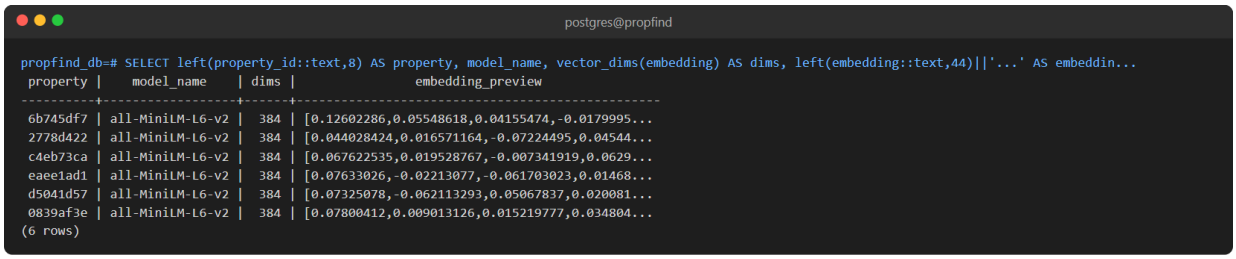


Fig. 6.3: Table property_embeddings: 384-dimension vectors (preview)

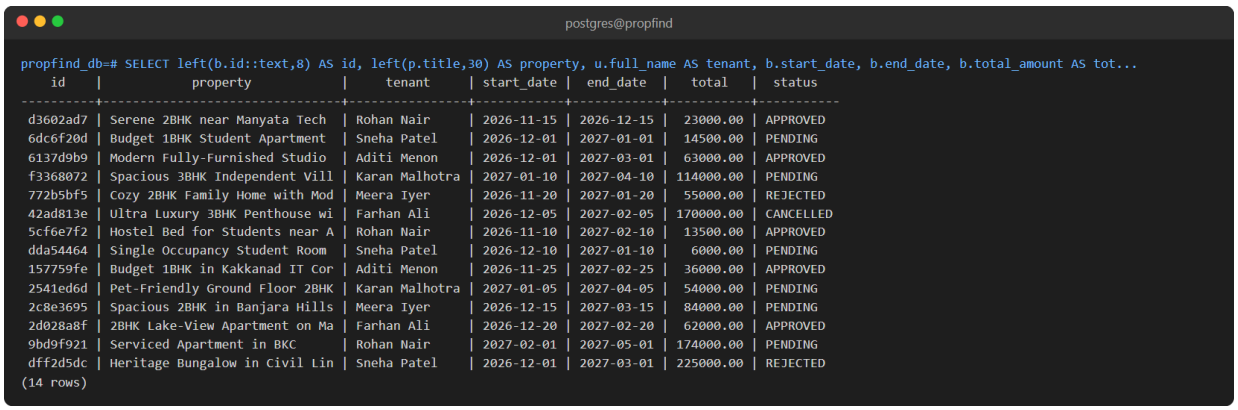


Fig. 6.4: Table bookings: all statuses

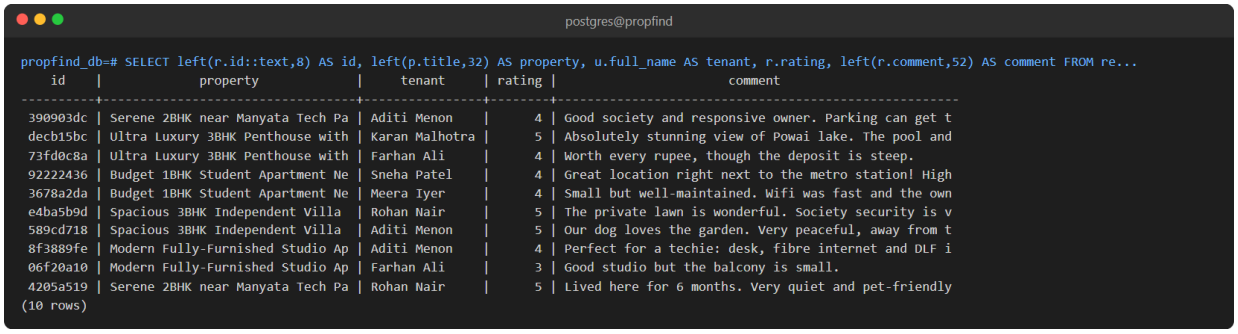


Fig. 6.5: Table reviews (first 10 rows)

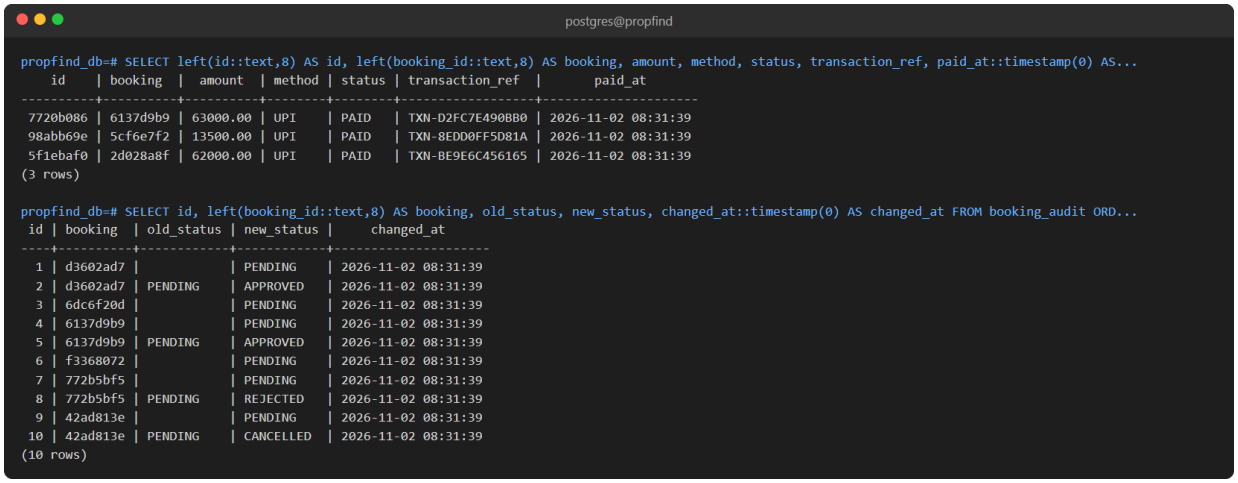


Fig. 6.6: Tables payments and booking_audit (the audit rows were written by a trigger)

6.3 DATA VOLUME

Table 6.1: Rows in the database

Table	Rows
users	13
properties	58 (in 15 cities)
property_embeddings	58
reviews	44
bookings	14
payments	3
booking_audit	22

7. QUERIES

7.1 SIMPLE QUERY WITH GROUP BY, AGGREGATE

```
-- B1. GROUP BY + COUNT / AVG / MIN / MAX: price statistics per city
SELECT city,
       COUNT(*)                            AS properties,
       ROUND(AVG(price_per_month), 2)      AS avg_rent,
       MIN(price_per_month)                AS min_rent,
       MAX(price_per_month)                AS max_rent
FROM properties
GROUP BY city
ORDER BY avg_rent DESC;

-- B2. HAVING: cities with at least 2 listings and average rent above 20,000
SELECT city, COUNT(*) AS properties, ROUND(AVG(price_per_month), 2) AS avg_rent
FROM properties
GROUP BY city
HAVING COUNT(*) >= 2 AND AVG(price_per_month) > 20000
ORDER BY avg_rent DESC;

-- B3. GROUP BY with SUM: revenue and count per booking status
SELECT status, COUNT(*) AS bookings, SUM(total_amount) AS total_value
FROM bookings
GROUP BY status
ORDER BY bookings DESC;
```

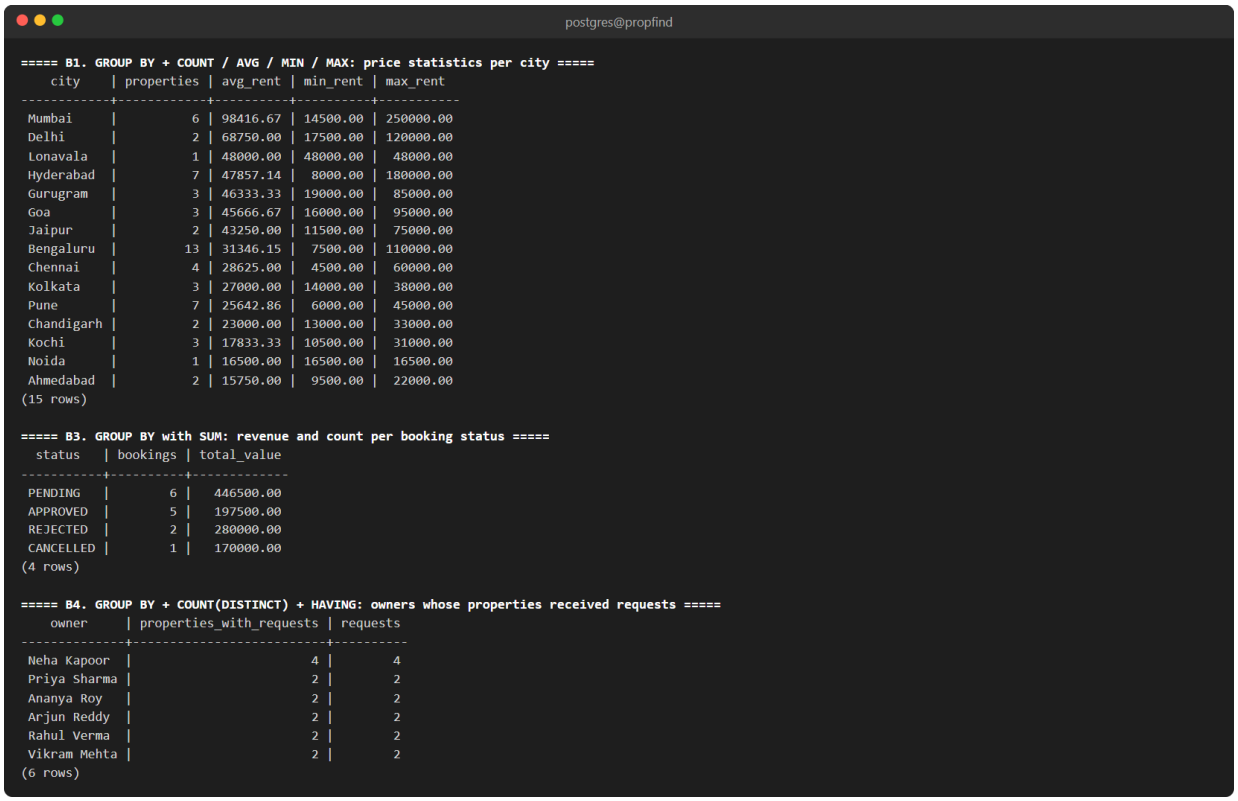


Fig. 7.1: GROUP BY with COUNT, AVG, MIN, MAX and SUM

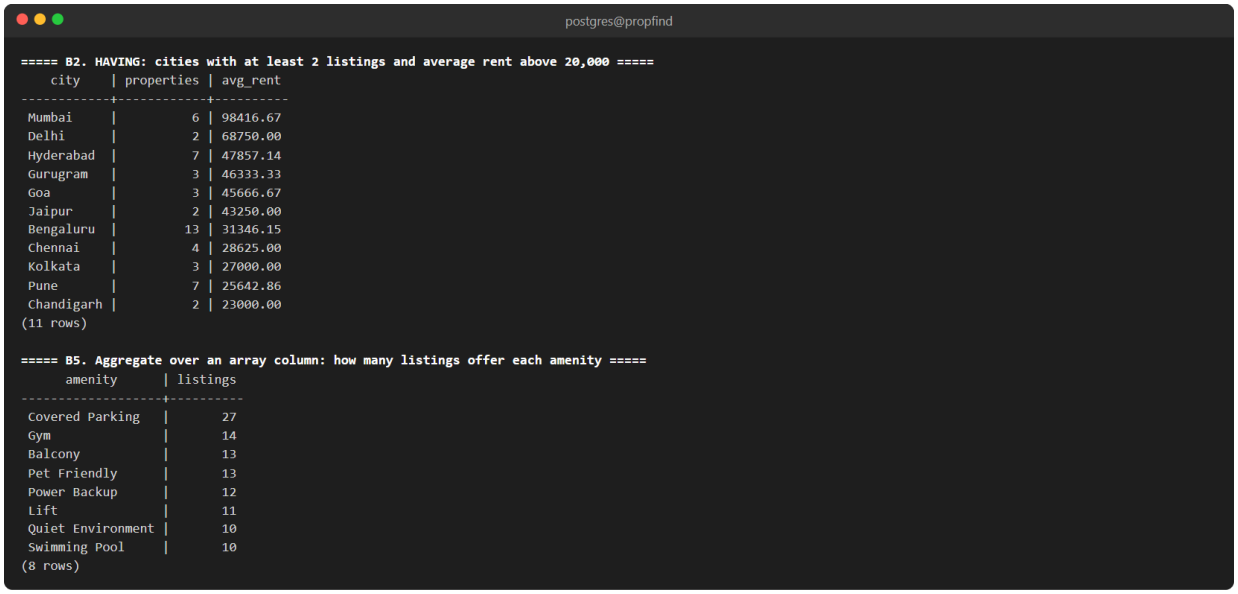


Fig. 7.2: HAVING and an aggregate over an array column

7.2 UPDATE OPERATION

```
-- F1. UPDATE with a subquery: raise rent 5% for owners with an APPROVED booking
BEGIN;
SELECT title, price_per_month AS before FROM properties WHERE city = 'Bengaluru' ORDER BY title;
UPDATE properties
SET price_per_month = ROUND(price_per_month * 1.05, 2)
WHERE id IN (SELECT property_id FROM bookings WHERE status = 'APPROVED');
SELECT title, price_per_month AS after, updated_at > created_at AS timestamp_trigger_fired
FROM properties WHERE id IN (SELECT property_id FROM bookings WHERE status = 'APPROVED');
ROLLBACK;
```

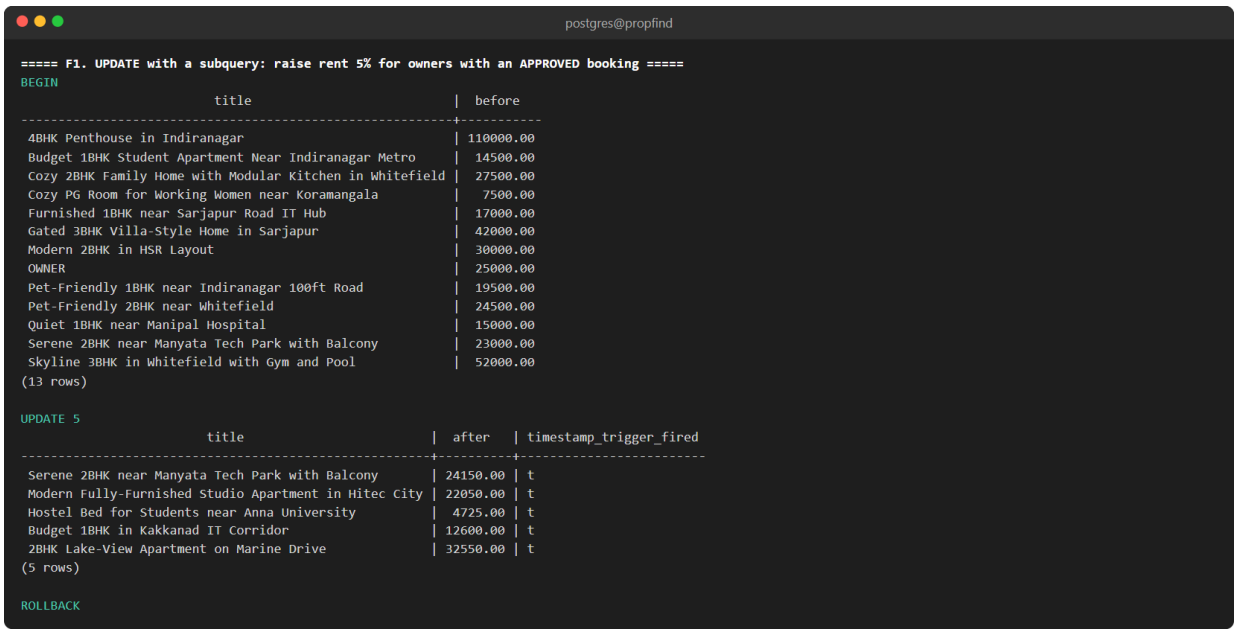


Fig. 7.3: UPDATE with a subquery; the timestamp trigger fires; the transaction is rolled back

7.3 DELETE OPERATION

```
-- F2. DELETE with a subquery: remove PENDING requests (audit table keeps the history)
BEGIN;
SELECT COUNT(*) AS pending_before FROM bookings WHERE status = 'PENDING';
DELETE FROM bookings WHERE status = 'PENDING' AND start_date > CURRENT_DATE;
```

```
SELECT COUNT(*) AS pending_after FROM bookings WHERE status = 'PENDING';
ROLLBACK;
SELECT COUNT(*) AS pending_restored FROM bookings WHERE status = 'PENDING';
```

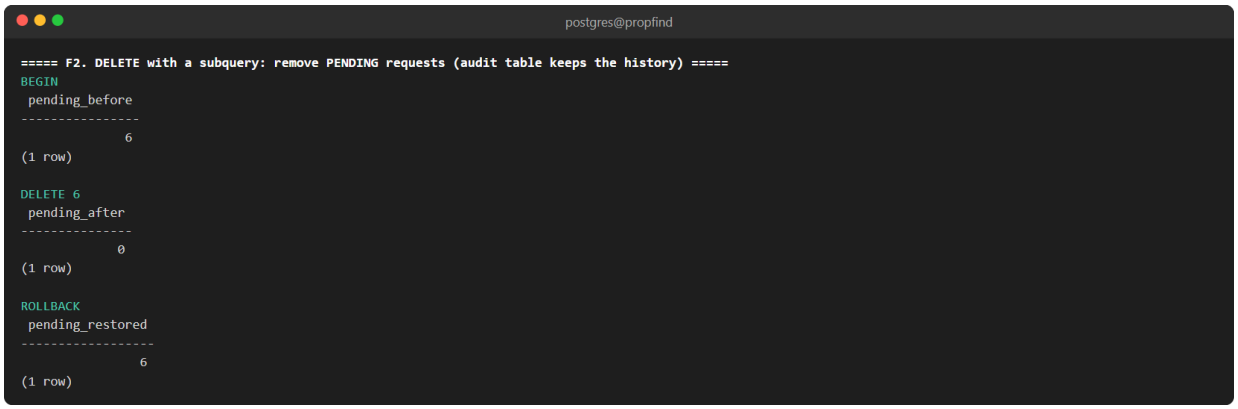


Fig. 7.4: DELETE with a condition, then ROLLBACK restores the rows

7.4 CORRELATED QUERY

```
-- D1. Correlated: properties priced above the average of THEIR OWN city
SELECT p.title, p.city, p.price_per_month
FROM properties p
WHERE p.price_per_month > (
    SELECT AVG(p2.price_per_month)
    FROM properties p2
    WHERE p2.city = p.city
)
ORDER BY p.city, p.price_per_month DESC;

-- D2. Correlated EXISTS: properties that have at least one 5-star review
SELECT p.title
FROM properties p
WHERE EXISTS (
    SELECT 1 FROM reviews r WHERE r.property_id = p.id AND r.rating = 5
);

-- D3. Correlated NOT EXISTS: properties nobody has requested yet
SELECT p.title, p.city
FROM properties p
WHERE NOT EXISTS (
    SELECT 1 FROM bookings b WHERE b.property_id = p.id
);
```



```
===== D1. Correlated: properties priced above the average of THEIR OWN city =====
                    title                    |    city    | price_per_month
---------------------------------------------+------------+-----------------
 2BHK with Balcony near SG Highway           | Ahmedabad  |        22000.00
 4BHK Penthouse in Indiranagar               | Bengaluru  |       110000.00
 Skyline 3BHK in Whitefield with Gym and Pool| Bengaluru  |        52000.00
 Gated 3BHK Villa-Style Home in Sarjapur     | Bengaluru  |        42000.00
 Independent Floor in Sector 21 Chandigarh   | Chandigarh |        33000.00
 4BHK Independent House in Besant Nagar      | Chennai    |        60000.00
 3BHK Family Flat in Adyar                   | Chennai    |        32000.00
 4BHK Independent Floor in Vasant Vihar      | Delhi      |       120000.00
 Luxury Villa with Private Pool in Assagao   | Goa        |        95000.00
 Designer 3BHK on Golf Course Extension Road | Gurugram   |        85000.00
 5BHK Villa with Pool in Jubilee Hills       | Hyderabad  |       180000.00
 Duplex 3BHK in Jubilee Hills                | Hyderabad  |        55000.00
 Heritage Bungalow in Civil Lines            | Jaipur     |        25000.00
 2BHK Lake-View Apartment on Marine Drive    | Kochi      |        31000.00
 3BHK Garden Apartment in Salt Lake          | Kolkata    |        38000.00
 Heritage 3BHK in Alipore                    | Kolkata    |        29000.00
 4BHK Sea-Facing Penthouse in Worli          | Mumbai     |       250000.00
 3BHK Sea-View Apartment in Bandra West      | Mumbai     |       150000.00
 Row House with Terrace in Baner             | Pune       |        45000.00
 Spacious 3BHK Independent Villa with Private Garden | Pune |       38000.00
 2BHK with Terrace in Koregaon Park          | Pune       |        34000.00
(21 rows)

===== D2. Correlated EXISTS: properties that have at least one 5-star review =====
                    title
---------------------------------------------------
 Serene 2BHK near Manyata Tech Park with Balcony
 Ultra Luxury 3BHK Penthouse with Private Terrace & Pool Access
 Spacious 3BHK Independent Villa with Private Garden
 Cozy 2BHK Family Home with Modular Kitchen in Whitefield
 Cozy PG Room for Working Women near Koramangala
 Heritage Style 2BHK Floor in Jaipur Old City
 Quiet 1BHK near Manipal Hospital
 1BHK Garden-Facing Flat in Porvorim
 Pet-friendly Ground Floor 2BHK in Velachery
 Pet-friendly 1BHK near Indiranagar 100Ft Road
 Pet-friendly 2BHK near Whitefield
 Fully Furnished 2BHK in Gachibowli
 3BHK Family Flat in Adyar
 Co-working Friendly 1BHK in Powai
 Gated 3BHK Villa-Style Home in Sarjapur
 5BHK Villa with Pool in Jubilee Hills
(16 rows)

===== D3. Correlated NOT EXISTS: properties nobody has requested yet =====
                    title                    |    city
---------------------------------------------+------------
 OWNER                                       | Bengaluru
 Cozy PG Room for Working Women near Koramangala | Bengaluru
 Single Room with Attached Bath near Kukatpally Metro | Hyderabad
 Newly Painted 1BHK in Maninagar Near Railway Station | Ahmedabad
 Studio near Infopark with Kitchenette       | Kochi
 Heritage Style 2BHK Floor in Jaipur Old City | Jaipur
 Compact Studio Near Sector 17 Plaza         | Chandigarh
 Affordable 2BHK near Salt Lake Sector V     | Kolkata
 Low-cost Bachelor Flat in Thane West        | Mumbai
 Quiet 1BHK near Manipal Hospital            | Bengaluru
 1BHK near Kondapur Metro for IT Professionals | Hyderabad
 1BHK Garden-Facing Flat in Porvorim         | Goa
 Furnished 1BHK near Sarjapur Road IT Hub    | Bengaluru
 Spacious 2BHK in Dwarka Sector 12 near Metro | Delhi
 Budget 2BHK for Families in Wakad           | Pune
 1BHK in Gurugram Sector 56 near Rapid Metro | Gurugram
 Pet-friendly 1BHK near Indiranagar 100Ft Road | Bengaluru
 Student Studio near Symbiosis, Viman Nagar  | Pune
 2BHK near Noida Sector 62 Metro             | Noida
 2BHK with Balcony near SG Highway           | Ahmedabad
 Pet-friendly 2BHK near Whitefield           | Bengaluru
 Quiet 2BHK near Aundh with Study Nook       | Pune
 2BHK Sea-Facing Apartment in Panjim         | Goa
 Fully Furnished 2BHK in Gachibowli          | Hyderabad
 Heritage 3BHK in Alipore                    | Kolkata
 Modern 2BHK in HSR Layout                   | Bengaluru
 3BHK Family Flat in Adyar                   | Chennai
 Independent Floor in Sector 21 Chandigarh   | Chandigarh
 Co-working Friendly 1BHK in Powai           | Mumbai
 2BHK with Terrace in Koregaon Park          | Pune
 Golf Course Road 2BHK Near Metro            | Gurugram
 3BHK Garden Apartment in Salt Lake          | Kolkata
 Gated 3BHK Villa-Style Home in Sarjapur     | Bengaluru
 Row House with Terrace in Baner             | Pune
 Duplex 3BHK in Jubilee Hills                | Hyderabad
 Skyline 3BHK in Whitefield with Gym and Pool| Bengaluru
 4BHK Independent House in Besant Nagar      | Chennai
 Farmhouse Retreat near Lonavala             | Lonavala
 Designer 3BHK on Golf Course Extension Road | Gurugram
 Luxury Villa with Private Pool in Assagao   | Goa
 4BHK Penthouse in Indiranagar               | Bengaluru
 4BHK Independent Floor in Vasant Vihar      | Delhi
 3BHK Sea-View Apartment in Bandra West      | Mumbai
 5BHK Villa with Pool in Jubilee Hills       | Hyderabad
 4BHK Sea-Facing Penthouse in Worli          | Mumbai
(45 rows)

===== D4. Correlated scalar subquery in SELECT: latest booking date per property =====
                    title                                                  | last_request
-----------------------------------------------------------------------------+--------------
 2BHK Lake-View Apartment on Marine Drive                                    | 2026-11-02
 Budget 1BHK in Kakkanad IT Corridor                                         | 2026-11-02
 Budget 1BHK Student Apartment Near Indiranagar Metro                        | 2026-11-02
 Cozy 2BHK Family Home with Modular Kitchen in Whitefield                    | 2026-11-02
 Heritage Bungalow in Civil Lines                                            | 2026-11-02
 Hostel Bed for Students near Anna University                                | 2026-11-02
 Modern Fully-Furnished Studio Apartment in Hitec City                       | 2026-11-02
 Pet-friendly Ground Floor 2BHK in Velachery                                 | 2026-11-02
 Serene 2BHK near Manyata Tech Park with Balcony                             | 2026-11-02
 Serviced Apartment in BKC                                                   | 2026-11-02
 Single Occupancy Student Room near Pune University                          | 2026-11-02
 Spacious 2BHK in Banjara Hills                                              | 2026-11-02
 Spacious 3BHK Independent Villa with Private Garden                         | 2026-11-02
 Ultra Luxury 3BHK Penthouse with Private Terrace & Pool Access              | 2026-11-02
 1BHK Garden-Facing Flat in Porvorim                                         | -
 1BHK in Gurugram Sector 56 near Rapid Metro                                 | -
 1BHK near Kondapur Metro for IT Professionals                               | -
 2BHK near Noida Sector 62 Metro                                             | -
 2BHK Sea-Facing Apartment in Panjim                                         | -
 2BHK with Balcony near SG Highway                                           | -
 2BHK with Terrace in Koregaon Park                                          | -
 3BHK Family Flat in Adyar                                                   | -
 3BHK Garden Apartment in Salt Lake                                          | -
 3BHK Sea-View Apartment in Bandra West                                      | -
 4BHK Independent Floor in Vasant Vihar                                      | -
 4BHK Independent House in Besant Nagar                                      | -
 4BHK Penthouse in Indiranagar                                               | -
 4BHK Sea-Facing Penthouse in Worli                                          | -
 5BHK Villa with Pool in Jubilee Hills                                       | -
 Affordable 2BHK near Salt Lake Sector V                                     | -
 Budget 2BHK for Families in Wakad                                           | -
 Compact Studio Near Sector 17 Plaza                                         | -
 Co-working Friendly 1BHK in Powai                                           | -
 Cozy PG Room for Working Women near Koramangala                             | -
 Designer 3BHK on Golf Course Extension Road                                 | -
 Duplex 3BHK in Jubilee Hills                                                | -
 Farmhouse Retreat near Lonavala                                             | -
 Fully Furnished 2BHK in Gachibowli                                          | -
 Furnished 1BHK near Sarjapur Road IT Hub                                    | -
 Gated 3BHK Villa-Style Home in Sarjapur                                     | -
 Golf Course Road 2BHK Near Metro                                            | -
 Heritage 3BHK in Alipore                                                    | -
 Heritage Style 2BHK Floor in Jaipur Old City                                | -
 Independent Floor in Sector 21 Chandigarh                                   | -
 Low-cost Bachelor Flat in Thane West                                        | -
 Luxury Villa with Private Pool in Assagao                                   | -
 Modern 3BHK in HSR Layout                                                   | -
 Newly Painted 1BHK in Maninagar Near Railway Station                        | -
 OWNER                                                                       | -
 Pet-friendly 1BHK near Indiranagar 100Ft Road                               | -
 Pet-friendly 2BHK near Whitefield                                           | -
 Quiet 1BHK near Manipal Hospital                                            | -
 Quiet 2BHK near Aundh with Study Nook                                       | -
 Row House with Terrace in Baner                                             | -
 Single Room with Attached Bath near Kukatpally Metro                        | -
 Skyline 3BHK in Whitefield with Gym and Pool                                | -
 Spacious 2BHK in Dwarka Sector 12 near Metro                                | -
 Student Studio near Symbiosis, Viman Nagar                                  | -
 Studio near Infopark with Kitchenette                                       | -
(59 rows)
```

Fig. 7.5: Correlated subqueries: comparison with the city average, EXISTS, NOT EXISTS and a scalar subquery

7.5 NESTED QUERY

```
-- C1. Scalar subquery: properties priced above the overall average
SELECT title, price_per_month
FROM properties
WHERE price_per_month > (SELECT AVG(price_per_month) FROM properties)
ORDER BY price_per_month DESC;
```

```sql
-- C2. IN subquery: tenants who have at least one APPROVED booking
SELECT full_name, email
FROM users
WHERE id IN (SELECT tenant_id FROM bookings WHERE status = 'APPROVED');

-- C3. Subquery in FROM (derived table): city with the highest average rent
SELECT city, avg_rent
FROM (
    SELECT city, AVG(price_per_month) AS avg_rent
    FROM properties
    GROUP BY city
) AS city_avg
ORDER BY avg_rent DESC
LIMIT 1;
```

```
postgres@propfind

===== C1. Scalar subquery: properties priced above the overall average =====
                           title                           | price_per_month
-----------------------------------------------------------+-----------------
 4BHK Sea-Facing Penthouse in Worli                        |       250000.00
 5BHK Villa with Pool in Jubilee Hills                     |       180000.00
 3BHK Sea-View Apartment in Bandra West                    |       150000.00
 4BHK Independent Floor in Vasant Vihar                    |       120000.00
 4BHK Penthouse in Indiranagar                             |       110000.00
 Luxury Villa with Private Pool in Assagao                 |        95000.00
 Ultra Luxury 3BHK Penthouse with Private Terrace & Pool Access |        85000.00
 Designer 3BHK on Golf Course Extension Road               |        85000.00
 Heritage Bungalow in Civil Lines                          |        75000.00
 4BHK Independent House in Besant Nagar                    |        60000.00
 Serviced Apartment in BKC                                 |        58000.00
 Duplex 3BHK in Jubilee Hills                              |        55000.00
 Skyline 3BHK in Whitefield with Gym and Pool              |        52000.00
 Farmhouse Retreat near Lonavala                           |        48000.00
 Row House with Terrace in Baner                           |        45000.00
 Gated 3BHK Villa-Style Home in Sarjapur                   |        42000.00
(16 rows)

===== C2. IN subquery: tenants who have at least one APPROVED booking =====
  full_name  |          email
-------------+-------------------------
 Rohan Nair  | rohan.nair@example.com
 Aditi Menon | aditi.menon@example.com
 Farhan Ali  | farhan.ali@example.com
(3 rows)

===== C3. Subquery in FROM (derived table): city with the highest average rent =====
  city  |      avg_rent
--------+--------------------
 Mumbai | 98416.666666666667
(1 row)

===== C4. ALL: properties more expensive than every Bengaluru listing =====
               title               |   city    | price_per_month
-----------------------------------+-----------+-----------------
 4BHK Independent Floor in Vasant Vihar | Delhi     |       120000.00
 3BHK Sea-View Apartment in Bandra West | Mumbai    |       150000.00
 5BHK Villa with Pool in Jubilee Hills  | Hyderabad |       180000.00
 4BHK Sea-Facing Penthouse in Worli     | Mumbai    |       250000.00
(4 rows)
```

Fig. 7.6: Nested subqueries: scalar, IN, derived table and ALL

7.6 JOIN QUERIES

```sql
-- A1. INNER JOIN (4 tables): every booking with property, tenant and owner
SELECT b.start_date, b.end_date, b.status,
       p.title         AS property,
       t.full_name     AS tenant,
       o.full_name     AS owner
FROM bookings b
JOIN properties p ON p.id = b.property_id
JOIN users t      ON t.id = b.tenant_id
JOIN users o      ON o.id = p.owner_id
ORDER BY b.start_date;

-- A2. LEFT JOIN: every property with its booking count (0 if never booked)
SELECT p.title, p.city, COUNT(b.id) AS bookings
FROM properties p
LEFT JOIN bookings b ON b.property_id = p.id
GROUP BY p.id, p.title, p.city
```

```
ORDER BY bookings DESC, p.title;

-- A3. LEFT JOIN + IS NULL (anti-join): properties that have no reviews
SELECT p.title
FROM properties p
LEFT JOIN reviews r ON r.property_id = p.id
WHERE r.id IS NULL;
```

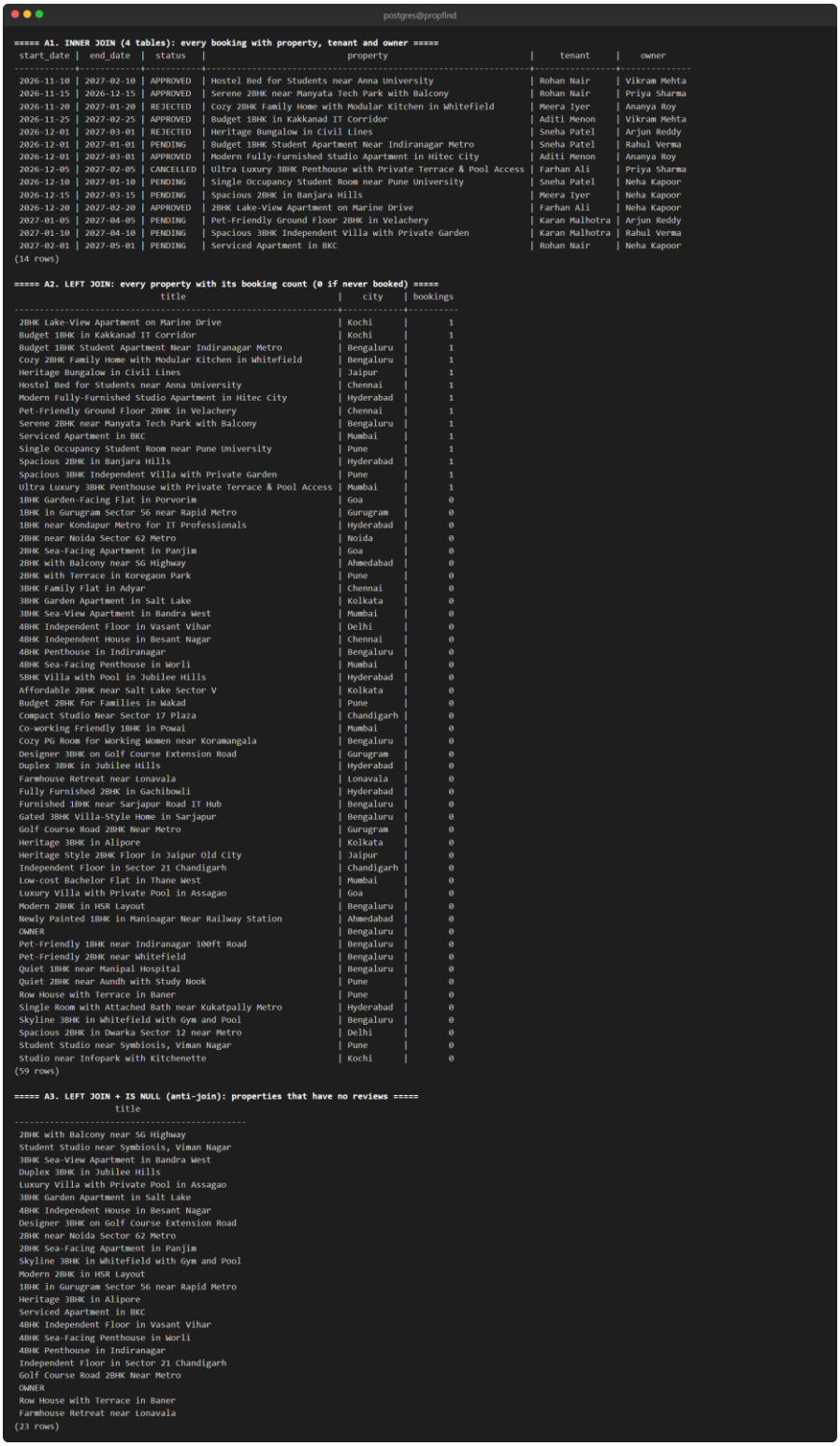


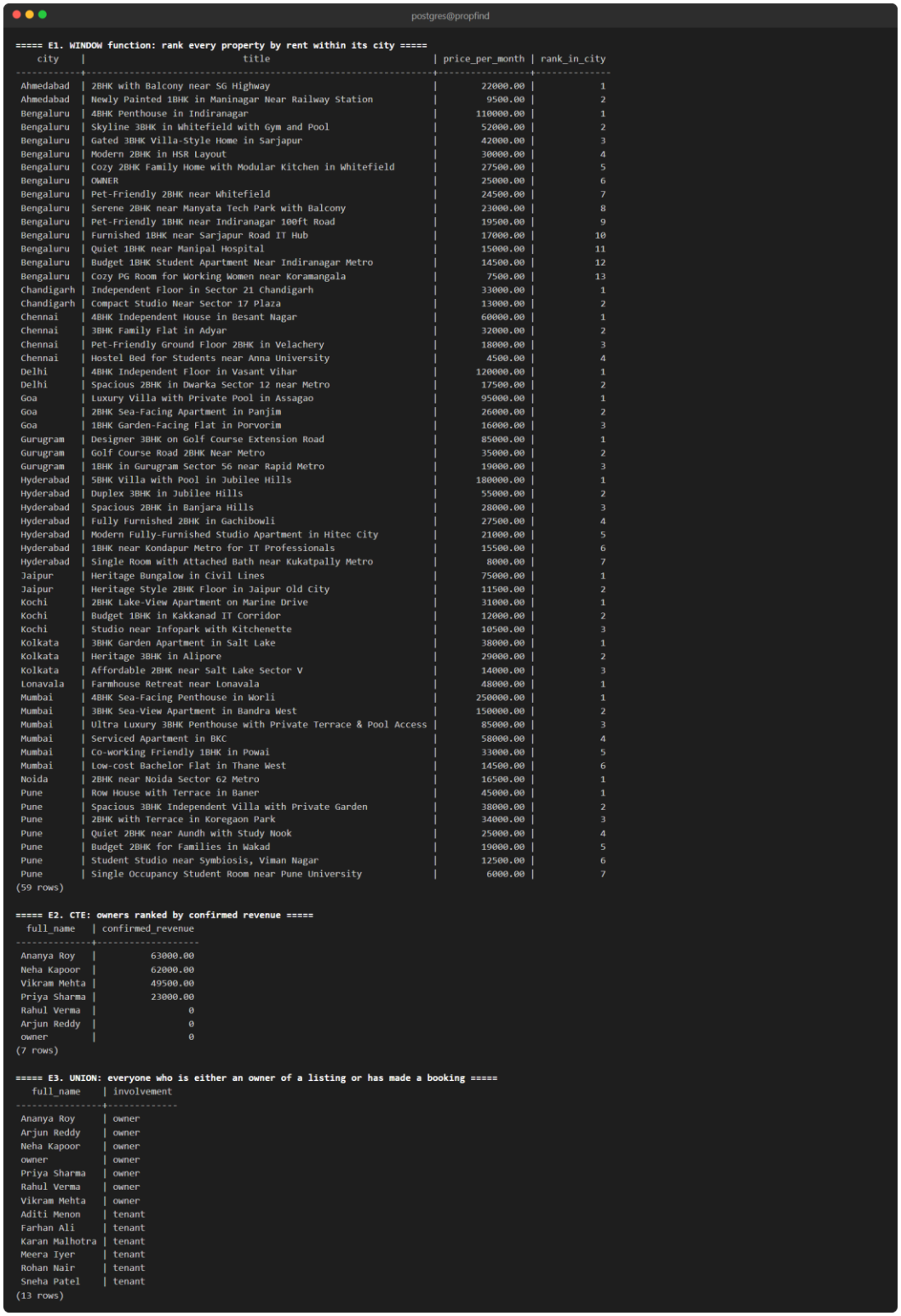
Fig. 7.7: INNER JOIN of four tables, LEFT JOIN and an anti-join

7.7 WINDOW FUNCTION, CTE AND SET OPERATION

```
-- E1. WINDOW function: rank every property by rent within its city
SELECT city, title, price_per_month,
       RANK() OVER (PARTITION BY city ORDER BY price_per_month DESC) AS rank_in_city
FROM properties
ORDER BY city, rank_in_city;

-- E2. CTE: owners ranked by confirmed revenue
WITH revenue AS (
    SELECT p.owner_id, SUM(b.total_amount) AS confirmed_revenue
    FROM bookings b
    JOIN properties p ON p.id = b.property_id
    WHERE b.status = 'APPROVED'
    GROUP BY p.owner_id
)
SELECT u.full_name, COALESCE(r.confirmed_revenue, 0) AS confirmed_revenue
FROM users u
LEFT JOIN revenue r ON r.owner_id = u.id
WHERE u.role = 'OWNER'
ORDER BY confirmed_revenue DESC;

-- E3. UNION: everyone who is either an owner of a listing or has made a booking
SELECT u.full_name, 'owner' AS involvement
FROM users u WHERE u.id IN (SELECT owner_id FROM properties)
UNION
SELECT u.full_name, 'tenant'
FROM users u WHERE u.id IN (SELECT tenant_id FROM bookings)
ORDER BY involvement, full_name;
```



```
postgres@propfind

===== E1. WINDOW function: rank every property by rent within its city =====
    city    |                           title                           | price_per_month | rank_in_city
------------+-----------------------------------------------------------+-----------------+--------------
 Ahmedabad  | 2BHK with Balcony near SG Highway                         |        22000.00 |            1
 Ahmedabad  | Newly Painted 1BHK in Maninagar Near Railway Station      |         9500.00 |            2
 Bengaluru  | 4BHK Penthouse in Indiranagar                             |       110000.00 |            1
 Bengaluru  | Skyline 3BHK in Whitefield with Gym and Pool              |        52000.00 |            2
 Bengaluru  | Gated 3BHK Villa-Style Home in Sarjapur                   |        42000.00 |            3
 Bengaluru  | Modern 2BHK in HSR Layout                                 |        30000.00 |            4
 Bengaluru  | Cozy 2BHK Family Home with Modular Kitchen in Whitefield  |        27500.00 |            5
 Bengaluru  | OWNER                                                     |        25000.00 |            6
 Bengaluru  | Pet-Friendly 2BHK near Whitefield                         |        24500.00 |            7
 Bengaluru  | Serene 2BHK near Manyata Tech Park with Balcony           |        23000.00 |            8
 Bengaluru  | Pet-Friendly 1BHK near Indiranagar 100ft Road             |        19500.00 |            9
 Bengaluru  | Furnished 1BHK near Sarjapur Road IT Hub                  |        17000.00 |           10
 Bengaluru  | Quiet 1BHK near Manipal Hospital                          |        15000.00 |           11
 Bengaluru  | Budget 1BHK Student Apartment Near Indiranagar Metro      |        14500.00 |           12
 Bengaluru  | Cozy PG Room for Working Women near Koramangala           |         7500.00 |           13
 Chandigarh | Independent Floor in Sector 21 Chandigarh                 |        33000.00 |            1
 Chandigarh | Compact Studio Near Sector 17 Plaza                       |        13000.00 |            2
 Chennai    | 4BHK Independent House in Besant Nagar                    |        60000.00 |            1
 Chennai    | 3BHK Family Flat in Adyar                                 |        32000.00 |            2
 Chennai    | Pet-Friendly Ground Floor 2BHK in Velachery               |        18000.00 |            3
 Chennai    | Hostel Bed for Students near Anna University              |         4500.00 |            4
 Delhi      | 4BHK Independent Floor in Vasant Vihar                    |       120000.00 |            1
 Delhi      | Spacious 2BHK in Dwarka Sector 12 near Metro              |        17500.00 |            2
 Goa        | Luxury Villa with Private Pool in Assagao                 |        95000.00 |            1
 Goa        | 2BHK Sea-Facing Apartment in Panjim                       |        26000.00 |            2
 Goa        | 1BHK Garden-Facing Flat in Porvorim                       |        16000.00 |            3
 Gurugram   | Designer 3BHK on Golf Course Extension Road               |        85000.00 |            1
 Gurugram   | Golf Course Road 2BHK Near Metro                          |        35000.00 |            2
 Gurugram   | 1BHK in Gurugram Sector 56 near Rapid Metro               |        19000.00 |            3
 Hyderabad  | 5BHK Villa with Pool in Jubilee Hills                     |       180000.00 |            1
 Hyderabad  | Duplex 3BHK in Jubilee Hills                              |        55000.00 |            2
 Hyderabad  | Spacious 2BHK in Banjara Hills                            |        28000.00 |            3
 Hyderabad  | Fully Furnished 2BHK in Gachibowli                        |        27500.00 |            4
 Hyderabad  | Modern Fully-Furnished Studio Apartment in Hitec City     |        21000.00 |            5
 Hyderabad  | 1BHK near Kondapur Metro for IT Professionals             |        15500.00 |            6
 Hyderabad  | Single Room with Attached Bath near Kukatpally Metro      |         8000.00 |            7
 Jaipur     | Heritage Bungalow in Civil Lines                          |        75000.00 |            1
 Jaipur     | Heritage Style 2BHK Floor in Jaipur Old City              |        11500.00 |            2
 Kochi      | 2BHK Lake-View Apartment on Marine Drive                  |        31000.00 |            1
 Kochi      | Budget 1BHK in Kakkanad IT Corridor                       |        12000.00 |            2
 Kochi      | Studio near Infopark with Kitchenette                     |        10500.00 |            3
 Kolkata    | 3BHK Garden Apartment in Salt Lake                        |        38000.00 |            1
 Kolkata    | Heritage 3BHK in Alipore                                  |        29000.00 |            2
 Kolkata    | Affordable 2BHK near Salt Lake Sector V                   |        14000.00 |            3
 Lonavala   | Farmhouse Retreat near Lonavala                           |        48000.00 |            1
 Mumbai     | 4BHK Sea-Facing Penthouse in Worli                        |       250000.00 |            1
 Mumbai     | 3BHK Sea-View Apartment in Bandra West                    |       150000.00 |            2
 Mumbai     | Ultra Luxury 3BHK Penthouse with Private Terrace & Pool Access |       85000.00 |            3
 Mumbai     | Serviced Apartment in BKC                                 |        58000.00 |            4
 Mumbai     | Co-working Friendly 1BHK in Powai                         |        33000.00 |            5
 Mumbai     | Low-cost Bachelor Flat in Thane West                      |        14500.00 |            6
 Noida      | 2BHK near Noida Sector 62 Metro                           |        16500.00 |            1
 Pune       | Row House with Terrace in Baner                           |        45000.00 |            1
 Pune       | Spacious 3BHK Independent Villa with Private Garden       |        38000.00 |            2
 Pune       | 2BHK with Terrace in Koregaon Park                        |        34000.00 |            3
 Pune       | Quiet 2BHK near Aundh with Study Nook                     |        25000.00 |            4
 Pune       | Budget 2BHK for Families in Wakad                         |        19000.00 |            5
 Pune       | Student Studio near Symbiosis, Viman Nagar                |        12500.00 |            6
 Pune       | Single Occupancy Student Room near Pune University        |         6000.00 |            7
(59 rows)

===== E2. CTE: owners ranked by confirmed revenue =====
  full_name   | confirmed_revenue
--------------+-------------------
 Ananya Roy   |          63000.00
 Neha Kapoor  |          62000.00
 Vikram Mehta |          49500.00
 Priya Sharma |          23000.00
 Rahul Verma  |                0
 Arjun Reddy  |                0
 owner        |                0
(7 rows)

===== E3. UNION: everyone who is either an owner of a listing or has made a booking =====
   full_name    | involvement
----------------+-------------
 Ananya Roy     | owner
 Arjun Reddy    | owner
 Neha Kapoor    | owner
 owner          | owner
 Priya Sharma   | owner
 Rahul Verma    | owner
 Vikram Mehta   | owner
 Aditi Menon    | tenant
 Farhan Ali     | tenant
 Karan Malhotra | tenant
 Meera Iyer     | tenant
 Rohan Nair     | tenant
 Sneha Patel    | tenant
(13 rows)
```

Fig. 7.8: RANK() window function, WITH query and UNION

7.8 VECTOR SIMILARITY AND HYBRID QUERIES

```
-- I1. Listings most similar to the Manyata Tech Park apartment (nearest neighbours)
SELECT p.title,
       ROUND((1 - (e.embedding <=> ref.embedding))::numeric, 3) AS cosine_similarity
FROM property_embeddings e
JOIN properties p ON p.id = e.property_id
```

```
CROSS JOIN (
    SELECT pe.embedding
    FROM property_embeddings pe JOIN properties pp ON pp.id = pe.property_id
    WHERE pp.title LIKE 'Serene 2BHK%'
) AS ref
WHERE p.title NOT LIKE 'Serene 2BHK%'
ORDER BY e.embedding <=> ref.embedding
LIMIT 3;

-- I2. Stored function fn_search_properties_semantic with a stored vector as the query
SELECT title, city, ROUND(similarity_score::numeric, 3) AS similarity
FROM fn_search_properties_semantic(
    (SELECT pe.embedding FROM property_embeddings pe JOIN properties pp ON pp.id =
pe.property_id
     WHERE pp.title LIKE 'Budget 1BHK Student%'),
    0.20, 3
);

-- I3. Hybrid: vector ranking + relational filters (city and price)
SELECT p.title, p.city, p.price_per_month,
       ROUND((1 - (e.embedding <=> ref.embedding))::numeric, 3) AS similarity
FROM property_embeddings e
JOIN properties p ON p.id = e.property_id
CROSS JOIN (
    SELECT pe.embedding FROM property_embeddings pe JOIN properties pp ON pp.id = pe.property_id
    WHERE pp.title LIKE 'Modern Fully-Furnished Studio%'
) AS ref
WHERE p.city = 'Bengaluru' AND p.price_per_month <= 30000
ORDER BY e.embedding <=> ref.embedding
LIMIT 3;
```

```
postgres@propfind

===== I1. Listings most similar to the Manyata Tech Park apartment (nearest neighbours) =====
                    title                     | cosine_similarity
----------------------------------------------+-------------------
 2BHK near Noida Sector 62 Metro              |             0.574
 2BHK with Balcony near SG Highway            |             0.556
 Spacious 3BHK Independent Villa with Private Garden |        0.541
(3 rows)

===== I2. Stored function fn_search_properties_semantic with a stored vector as the query =====
                    title                     |   city    | similarity
----------------------------------------------+-----------+------------
 Budget 1BHK Student Apartment Near Indiranagar Metro | Bengaluru |      1.000
 4BHK Penthouse in Indiranagar                | Bengaluru |      0.710
 Pet-Friendly 1BHK near Indiranagar 100ft Road | Bengaluru |      0.701
(3 rows)
```

Fig. 7.9: Nearest neighbours with the <=> operator and the semantic search function

```
postgres@propfind

===== I3. Hybrid: vector ranking + relational filters (city and price) =====
                    title                     |   city    | price_per_month | similarity
----------------------------------------------+-----------+-----------------+------------
 Modern 2BHK in HSR Layout                    | Bengaluru |        30000.00 |      0.434
 Cozy 2BHK Family Home with Modular Kitchen in Whitefield | Bengaluru |    27500.00 |      0.424
 Furnished 1BHK near Sarjapur Road IT Hub     | Bengaluru |        17000.00 |      0.411
(3 rows)

===== I4. Vector index in use: the HNSW index exists on property_embeddings =====
         indexname          |                                                    indexdef                                                    ...
----------------------------+----------------------------------------------------------------------------------------------------------------...
 idx_property_embeddings_hnsw | CREATE INDEX idx_property_embeddings_hnsw ON public.property_embeddings USING hnsw (embedding vector_cosine_ops) WI...
(1 row)
```

Fig. 7.10: Hybrid query (vector ranking with relational filters) and the HNSW index

7.9 FULL-TEXT SEARCH

```
-- J1. Full-text search: listings mentioning quiet AND garden (ranked)
SELECT title,
       ROUND(ts_rank(to_tsvector('english', title || ' ' || description),
                     to_tsquery('english', 'quiet & garden'))::numeric, 4) AS rank
FROM properties
WHERE to_tsvector('english', title || ' ' || description) @@ to_tsquery('english', 'quiet &
```

```
garden')
ORDER BY rank DESC;
```

```
postgres@propfind

===== J1. Full-text search: listings mentioning quiet AND garden (ranked) =====
                      title                      |  rank
-------------------------------------------------+--------
 Spacious 3BHK Independent Villa with Private Garden | 0.1036
 3BHK Garden Apartment in Salt Lake              | 0.1012
 1BHK Garden-Facing Flat in Porvorim             | 0.0000
(3 rows)

===== K1. LEFT JOIN payments: approved bookings and whether they are paid =====
                id                |                      title                      | total_amount | payment
----------------------------------+-------------------------------------------------+--------------+---------
 891c727e-66fc-44d4-ab70-b9a733670aca | Serene 2BHK near Manyata Tech Park with Balcony |     23000.00 | UNPAID
 078834bb-e544-43b8-ab7e-1e5f38e173e1 | Modern Fully-Furnished Studio Apartment in Hitec City |     63000.00 | PAID
 4d3bfb47-251a-4f86-b8b3-f0a64dabb736 | Hostel Bed for Students near Anna University |     13500.00 | PAID
 26c7f614-b5ee-4aca-a58b-25909c2cd77c | Budget 1BHK in Kakkanad IT Corridor |     36000.00 | UNPAID
 72d725c5-c92f-4cf9-99e4-44a3f70a91dd | 2BHK Lake-View Apartment on Marine Drive |     62000.00 | PAID
(5 rows)
```

Fig. 7.11: Full-text search with tsvector and tsquery, and a payments query

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

8.1 STORED PROCEDURES OR FUNCTIONS

The database logic is written as PL/pgSQL functions. PostgreSQL functions are used in place of CALL-style stored procedures; the multi-step booking transaction is executed by the application service inside BEGIN and COMMIT with a row lock (Section 8.3).

Table 8.1: Stored functions

Function	Returns	Purpose
fn_update_timestamp()	trigger	Sets updated_at on every update
fn_check_booking_conflict(property, start, end, exclude_booking)	boolean	True if an APPROVED booking overlaps the dates (uses OVERLAPS)
fn_search_properties_semantic(vector, threshold, count, city, max_price, bedrooms)	table	Cosine-similarity search with relational filters
fn_trg_prevent_double_booking()	trigger	Rejects overlapping APPROVED bookings
fn_trg_validate_booking_transition()	trigger	Allows only valid booking status changes
fn_trg_payment_stamp_and_validate()	trigger	Validates a payment and stamps paid_at
fn_trg_audit_booking_status()	trigger	Writes the audit log
fn_trg_refund_on_cancel()	trigger	Refunds the payment when a paid booking is cancelled

db/schema.sql (timestamp function and booking conflict function)

```sql
-- 1. Function: Auto-update timestamp
CREATE OR REPLACE FUNCTION fn_update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Function: Check date conflict for bookings (Prevents Double Booking)
CREATE OR REPLACE FUNCTION fn_check_booking_conflict(
    p_property_id UUID,
    p_start_date DATE,
    p_end_date DATE,
    p_exclude_booking_id UUID DEFAULT NULL
)
RETURNS BOOLEAN AS $$
DECLARE
    conflict_count INT;
BEGIN
    SELECT COUNT(*)
    INTO conflict_count
    FROM bookings
    WHERE property_id = p_property_id
      AND status = 'APPROVED'
      AND (id <> p_exclude_booking_id OR p_exclude_booking_id IS NULL)
      AND (
          (start_date, end_date) OVERLAPS (p_start_date, p_end_date)
      );

    RETURN (conflict_count > 0);
END;
$$ LANGUAGE plpgsql;
```

Fig. 8.1: fn_update_timestamp and fn_check_booking_conflict

db/schema.sql (fn_search_properties_semantic: pgvector cosine search)

```sql
CREATE OR REPLACE FUNCTION fn_search_properties_semantic(
    p_query_vector vector(384),
    p_match_threshold DOUBLE PRECISION DEFAULT 0.20,
    p_match_count INT DEFAULT 10,
    p_city VARCHAR DEFAULT NULL,
    p_max_price NUMERIC DEFAULT NULL,
    p_bedrooms INT DEFAULT NULL
)
RETURNS TABLE (
    id UUID,
    title VARCHAR,
    description TEXT,
    property_type VARCHAR,
    price_per_month NUMERIC,
    bedrooms INT,
    bathrooms INT,
    area_sqft INT,
    address TEXT,
    city VARCHAR,
    status property_status,
    amenities TEXT[],
    image_url TEXT,
    owner_name VARCHAR,
    avg_rating NUMERIC,
    similarity_score DOUBLE PRECISION
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        vpc.id,
        vpc.title,
        vpc.description,
        vpc.property_type,
        vpc.price_per_month,
        vpc.bedrooms,
        vpc.bathrooms,
        vpc.area_sqft,
        vpc.address,
        vpc.city,
        vpc.status,
        vpc.amenities,
        vpc.image_url,
        vpc.owner_name,
        vpc.avg_rating,
        -- Cosine distance (<=>) ranges from 0 (identical) to 2 (opposite)
        -- Similarity score is normalized to [0, 1]
        (1 - (pe.embedding <=> p_query_vector)) AS similarity_score
    FROM property_embeddings pe
    JOIN view_property_catalog vpc ON pe.property_id = vpc.id
    WHERE (1 - (pe.embedding <=> p_query_vector)) >= p_match_threshold
      AND (p_city IS NULL OR LOWER(vpc.city) = LOWER(p_city))
      AND (p_max_price IS NULL OR vpc.price_per_month <= p_max_price)
      AND (p_bedrooms IS NULL OR vpc.bedrooms = p_bedrooms)
      AND vpc.status = 'AVAILABLE'
    ORDER BY pe.embedding <=> p_query_vector ASC
    LIMIT p_match_count;
END;
$$ LANGUAGE plpgsql;

-- ------------------------------------------------------------------------
```

Fig. 8.2: fn_search_properties_semantic: pgvector cosine search with filters

postgres@propfind

```
propfind_db=# \df fn_*
                                                                                                                                  ...
 Schema |                Name                |                                                                                                  ...
--------+------------------------------------+-----------------------------------------------------------------------------------------------...
 public | fn_check_booking_conflict          | boolean                                                                                          ...
 public | fn_search_properties_semantic      | TABLE(id uuid, title character varying, description text, property_type character varying,...
 public | fn_trg_audit_booking_status        | trigger                                                                                          ...
 public | fn_trg_payment_stamp_and_validate  | trigger                                                                                          ...
 public | fn_trg_prevent_double_booking      | trigger                                                                                          ...
 public | fn_trg_refund_on_cancel            | trigger                                                                                          ...
 public | fn_trg_validate_booking_transition | trigger                                                                                          ...
 public | fn_update_timestamp                | trigger                                                                                          ...
(8 rows)
```

Fig. 8.3: Functions installed in the database (\df fn_*)

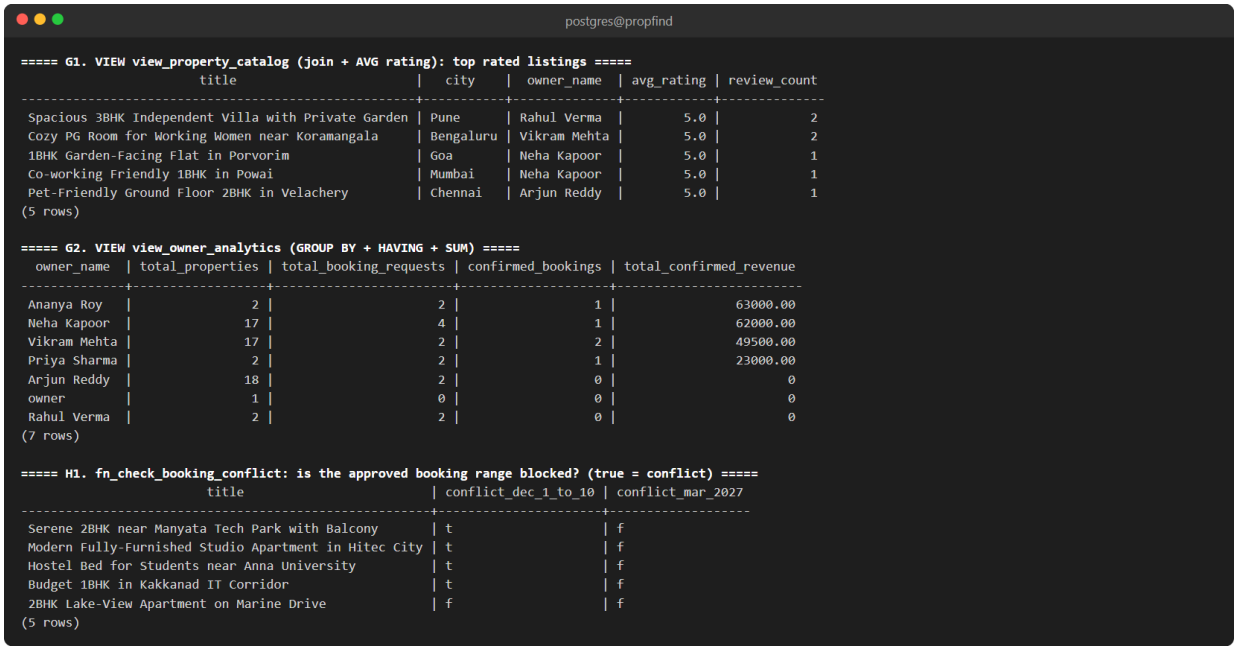


Fig. 8.4: Views and fn_check_booking_conflict in use

8.2 TRIGGERS

Table 8.2: Triggers

Trigger	Table	Event	Effect
trg_properties_timestamp	properties	BEFORE UPDATE	Maintains updated_at
trg_bookings_timestamp	bookings	BEFORE UPDATE	Maintains updated_at
trg_validate_booking_overlap	bookings	BEFORE INSERT OR UPDATE	Raises an exception if an APPROVED booking would overlap another
trg_validate_booking_transition	bookings	BEFORE UPDATE OF status	Allows only PENDING to APPROVED / REJECTED / CANCELLED and APPROVED to CANCELLED
trg_payment_stamp_and_validate	payments	BEFORE INSERT OR UPDATE	Payment only for an APPROVED booking and for the exact total; stamps paid_at
trg_audit_booking_status	bookings	AFTER INSERT OR UPDATE OF status	Writes old and new status and the acting user to booking_audit
trg_refund_on_cancel	bookings	AFTER UPDATE OF status	Marks a PAID payment REFUNDED when the booking is cancelled

db/schema.sql (double-booking trigger and booking state machine)

```
-- Trigger 3: Prevent overlapping confirmed bookings via trigger
CREATE OR REPLACE FUNCTION fn_trg_prevent_double_booking()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'APPROVED' THEN
        IF fn_check_booking_conflict(NEW.property_id, NEW.start_date, NEW.end_date, NEW.id) THEN
            RAISE EXCEPTION 'Double booking conflict: Property % is already booked for date range % to %',
                NEW.property_id, NEW.start_date, NEW.end_date;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validate_booking_overlap
BEFORE INSERT OR UPDATE ON bookings
FOR EACH ROW
EXECUTE FUNCTION fn_trg_prevent_double_booking();

-- Trigger 4: Enforce the booking state machine at the database level
--   PENDING  -> APPROVED | REJECTED | CANCELLED
--   APPROVED -> CANCELLED
--   REJECTED / CANCELLED are terminal
CREATE OR REPLACE FUNCTION fn_trg_validate_booking_transition()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = OLD.status THEN
        RETURN NEW;
    END IF;
    IF (OLD.status = 'PENDING'  AND NEW.status IN ('APPROVED', 'REJECTED', 'CANCELLED'))
    OR (OLD.status = 'APPROVED' AND NEW.status = 'CANCELLED') THEN
        RETURN NEW;
    END IF;
    RAISE EXCEPTION 'Invalid booking status change: % -> %', OLD.status, NEW.status;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_validate_booking_transition ON bookings;
CREATE TRIGGER trg_validate_booking_transition
BEFORE UPDATE OF status ON bookings
FOR EACH ROW
EXECUTE FUNCTION fn_trg_validate_booking_transition();
```

Fig. 8.5: Double-booking trigger and booking state machine

db/schema.sql (audit trigger and automatic refund trigger)

```sql
-- Trigger 6: Audit every booking status change (AFTER INSERT / UPDATE OF status)
--   The application sets app.user_id (set_config, transaction-local) so changed_by is recorded.
CREATE OR REPLACE FUNCTION fn_trg_audit_booking_status()
RETURNS TRIGGER AS $$
DECLARE
    actor UUID;
BEGIN
    BEGIN
        actor := NULLIF(current_setting('app.user_id', true), '')::uuid;
    EXCEPTION WHEN invalid_text_representation THEN
        actor := NULL;
    END;

    IF TG_OP = 'INSERT' THEN
        INSERT INTO booking_audit (booking_id, old_status, new_status, changed_by)
        VALUES (NEW.id, NULL, NEW.status, COALESCE(actor, NEW.tenant_id));
    ELSIF NEW.status IS DISTINCT FROM OLD.status THEN
        INSERT INTO booking_audit (booking_id, old_status, new_status, changed_by)
        VALUES (NEW.id, OLD.status, NEW.status, actor);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_booking_status
AFTER INSERT OR UPDATE OF status ON bookings
FOR EACH ROW
EXECUTE FUNCTION fn_trg_audit_booking_status();

-- Trigger 7: Refund the payment automatically when a paid booking is cancelled
CREATE OR REPLACE FUNCTION fn_trg_refund_on_cancel()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'CANCELLED' AND OLD.status <> 'CANCELLED' THEN
        UPDATE payments SET status = 'REFUNDED'
        WHERE booking_id = NEW.id AND status = 'PAID';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_refund_on_cancel
AFTER UPDATE OF status ON bookings
FOR EACH ROW
EXECUTE FUNCTION fn_trg_refund_on_cancel();
```

Fig. 8.6: Audit trigger and automatic refund trigger

postgres@propfind

```
propfind_db=# SELECT tgname AS trigger, tgrelid::regclass AS on_table, ...
trg_properties_timestamp | properties | CREATE TRIGGER trg_properties_timestamp BEFORE UPDATE ON public.properties FOR EACH ROW EXECUTE F...
trg_audit_booking_status | bookings | CREATE TRIGGER trg_audit_booking_status AFTER INSERT OR UPDATE OF status ON public.bookings FOR EAC...
trg_bookings_timestamp | bookings | CREATE TRIGGER trg_bookings_timestamp BEFORE UPDATE ON public.bookings FOR EACH ROW EXECUTE FUNCTION ...
trg_refund_on_cancel | bookings | CREATE TRIGGER trg_refund_on_cancel AFTER UPDATE OF status ON public.bookings FOR EACH ROW EXECUTE FUNC...
trg_validate_booking_overlap | bookings | CREATE TRIGGER trg_validate_booking_overlap BEFORE INSERT OR UPDATE ON public.bookings FOR EACH...
trg_validate_booking_transition | bookings | CREATE TRIGGER trg_validate_booking_transition BEFORE UPDATE OF status ON public.bookings FO...
trg_payment_stamp_and_validate | payments | CREATE TRIGGER trg_payment_stamp_and_validate BEFORE INSERT OR UPDATE ON public.payments FOR ...
```

Fig. 8.7: All triggers installed in the database

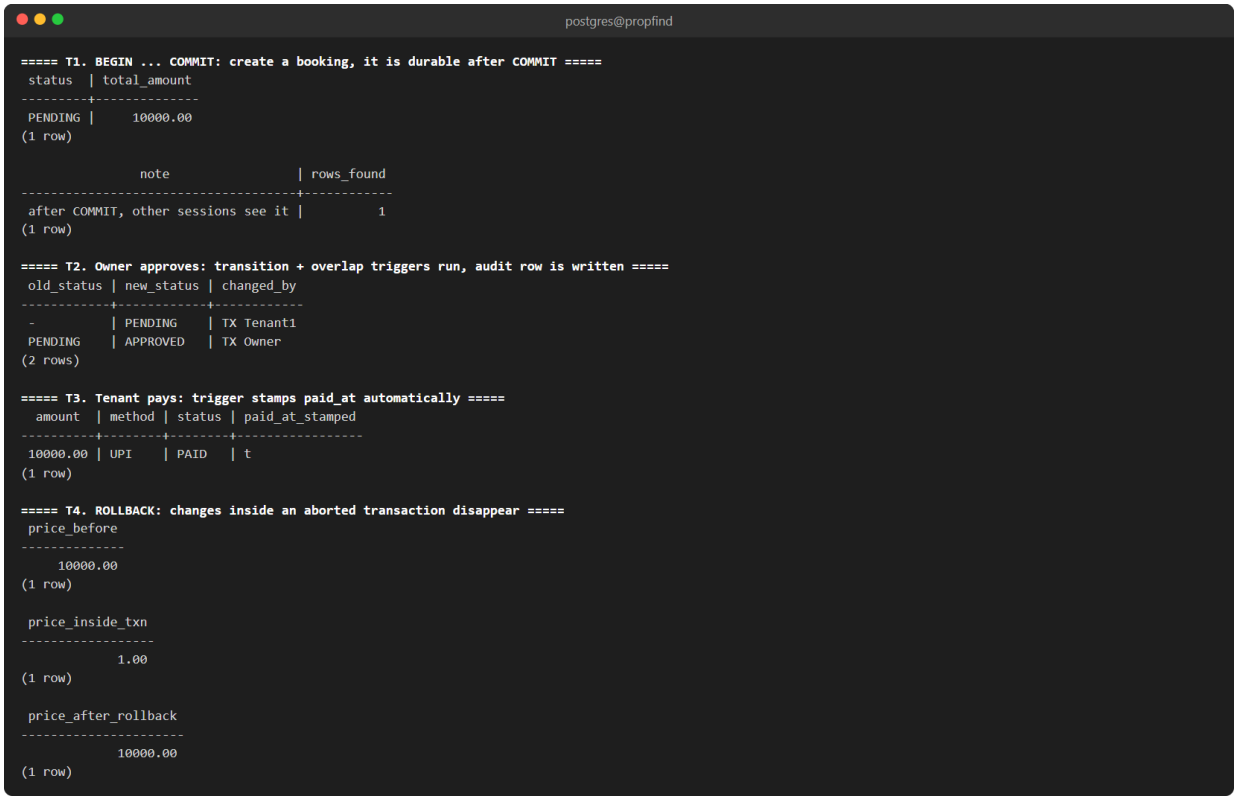


Fig. 8.8: Triggers in action: audit row written, payment timestamp stamped, ROLLBACK

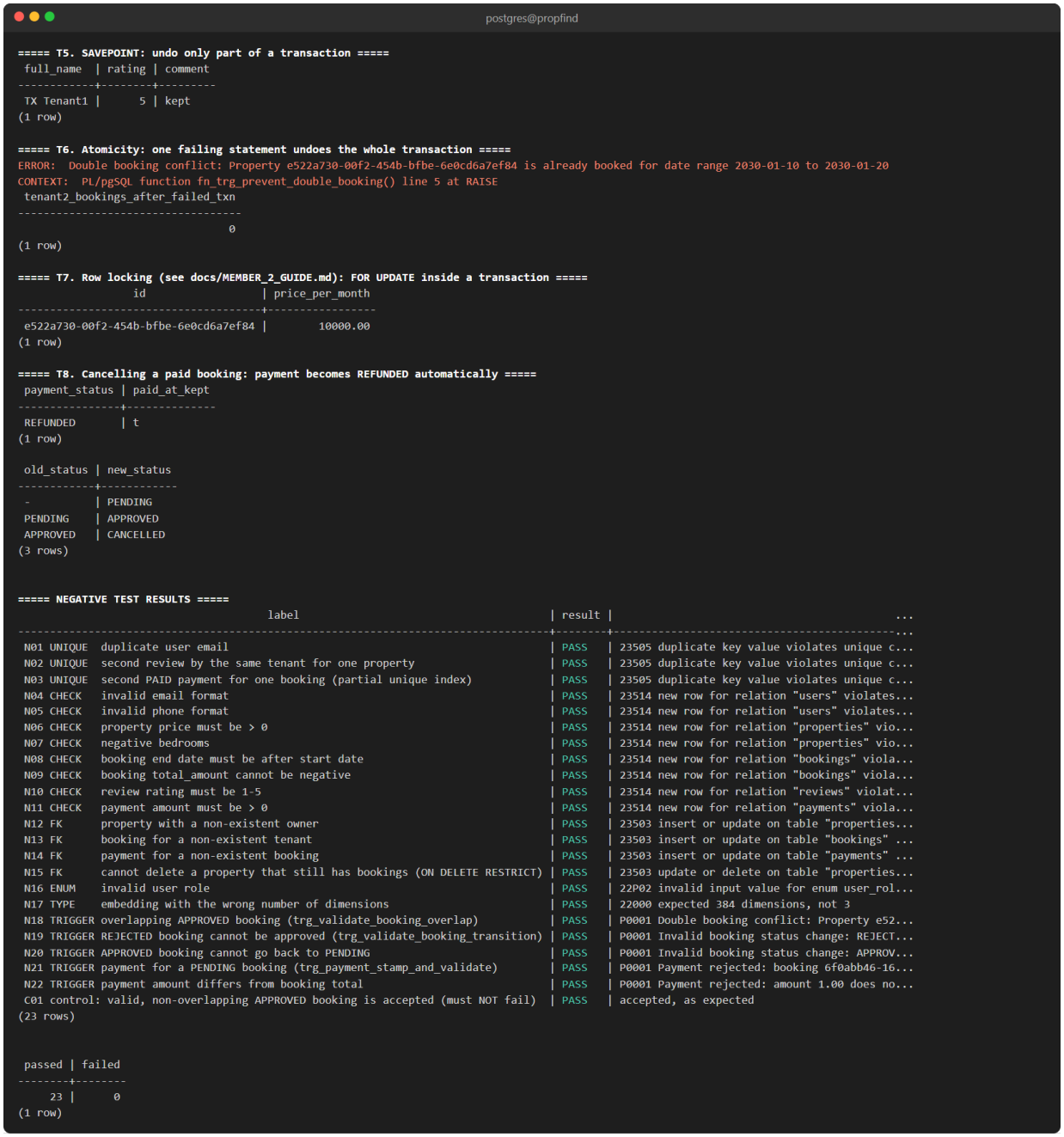


Fig. 8.9: SAVEPOINT, atomicity of a failed transaction and the automatic refund

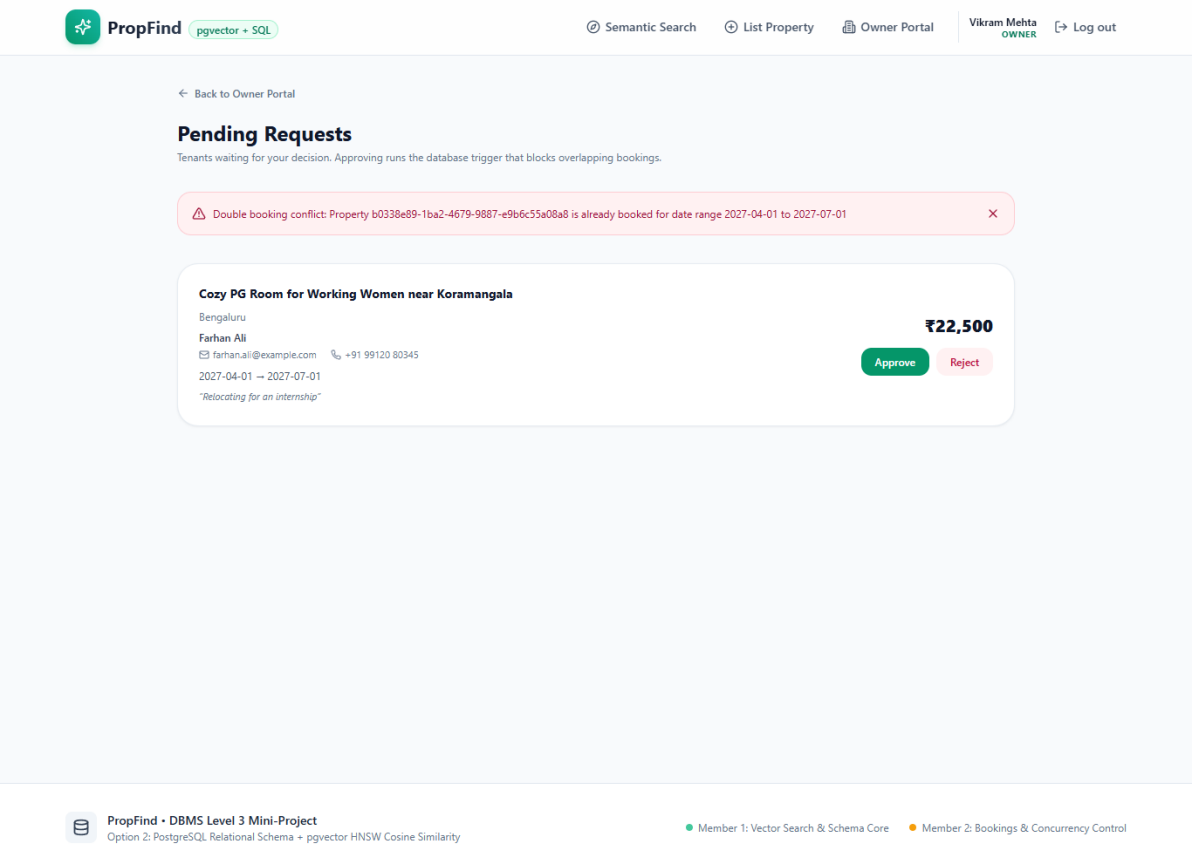


Fig. 8.10: The overlap trigger rejects an approval and the message is shown in the Owner Portal

8.3 TRANSACTIONS AND CONCURRENCY CONTROL

Creating a booking is one ACID transaction. The property row is locked with SELECT ... FOR UPDATE so concurrent requests for the same property wait for each other, then the conflict function is called, the total is computed and the booking is inserted. The same transaction sets the acting user for the audit trigger. If any step fails the whole transaction is rolled back.

lib/services/booking-service.ts (transactional booking with row lock)

```
export async function createBookingRequest(
  data: BookingRequestInput
): Promise<{ success: boolean; bookingId?: string; message: string }> {
  const start = parseDate('Start date', data.start_date);
  const end = parseDate('End date', data.end_date);
  if (end <= start) throw new Error('End date must be after the start date');

  const today = new Date();
  today.setUTCHours(0, 0, 0, 0);
  if (start < today) throw new Error('Start date cannot be in the past');

  return await withTransaction(async (client) => {
    await client.query(`SELECT set_config('app.user_id', $1, true)`, [data.tenant_id]);

    // Step 1: lock the property row so concurrent requests are serialised
    const propCheck = await client.query(
      `SELECT id, price_per_month, status, owner_id
       FROM properties
       WHERE id = $1
       FOR UPDATE`,
      [data.property_id]
    );

    if (propCheck.rows.length === 0) throw new Error('Property not found');

    const property = propCheck.rows[0];
    if (property.owner_id === data.tenant_id) {
      throw new Error('You cannot book your own property');
    }
    if (property.status !== 'AVAILABLE') {
      throw new Error(`Property is currently ${property.status} and cannot be booked`);
    }

    // Step 2: date conflict check against already APPROVED bookings (stored function)
    const conflictCheck = await client.query(
      `SELECT fn_check_booking_conflict($1, $2::date, $3::date) AS has_conflict`,
      [data.property_id, data.start_date, data.end_date]
    );
    if (conflictCheck.rows[0]?.has_conflict) {
      throw new Error('Property has an overlapping confirmed booking for these dates');
    }

    // Step 3: a tenant may not hold two live requests for overlapping dates on one property
    const dup = await client.query(
      `SELECT 1 FROM bookings
       WHERE property_id = $1 AND tenant_id = $2
         AND status IN ('PENDING', 'APPROVED')
         AND (start_date, end_date) OVERLAPS ($3::date, $4::date)
       LIMIT 1`,
      [data.property_id, data.tenant_id, data.start_date, data.end_date]
    );
    if (dup.rows.length > 0) {
      throw new Error('You already have an active request for these dates on this property');
    }

    // Step 4: total = price_per_month * months (30-day blocks, minimum 1)
    const days = Math.round((end.getTime() - start.getTime()) / 86_400_000);
    const months = Math.max(1, Math.round(days / 30));
    const totalAmount = Number(property.price_per_month) * months;

    // Step 5: insert as PENDING
    const insertRes = await client.query(
```

Fig. 8.11: Transactional booking creation with a row lock (lib/services/booking-service.ts)

8.4 INTEGRITY TEST RESULTS

db/transactions_tests.sql runs 22 negative tests, each of which must be rejected by the database (UNIQUE, CHECK, FOREIGN KEY, ENUM, vector dimension and trigger violations), plus one positive control that must succeed. The script exits with an error if any check fails. Result: 23 passed, 0 failed.

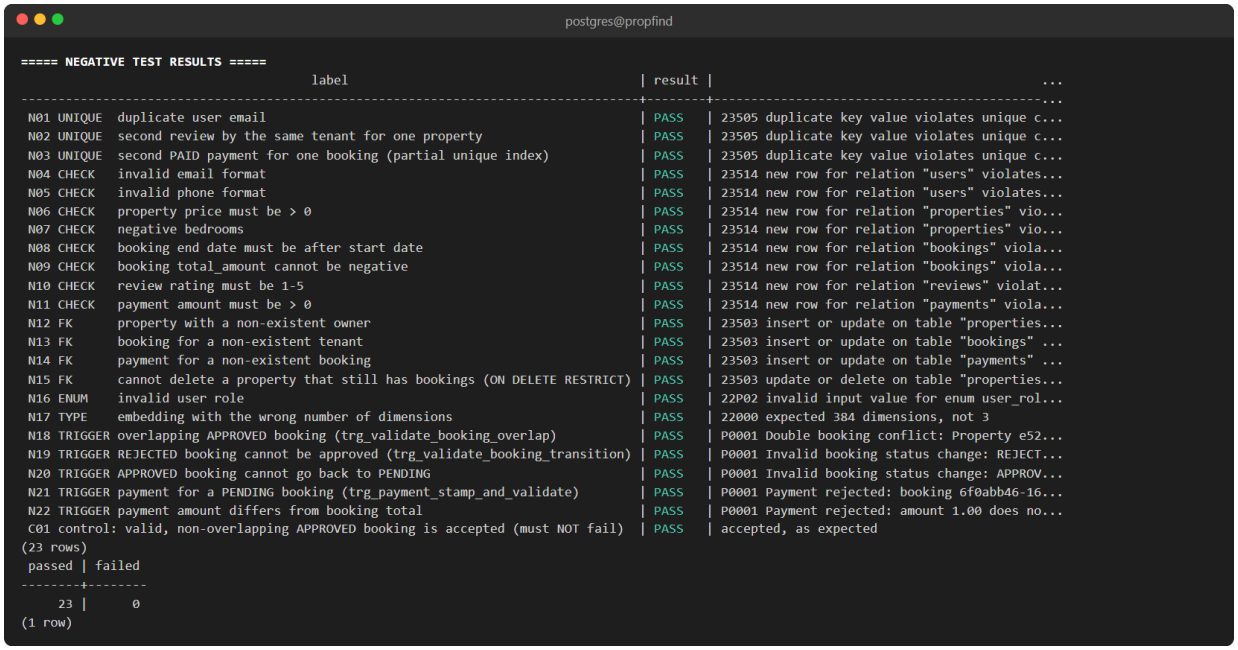


Fig. 8.12: Integrity tests: 22 rejected statements and one accepted control, 23 of 23 passed

9. FRONT END DEVELOPMENT

9.1 ARCHITECTURE

The interface is a Next.js 14 application. Pages call route handlers under /api, which call a service layer that runs parameterised SQL through a connection pool. Role checks happen in middleware (for pages) and again in every route handler. Database credentials and the JWT secret come only from environment variables (.env is not committed).

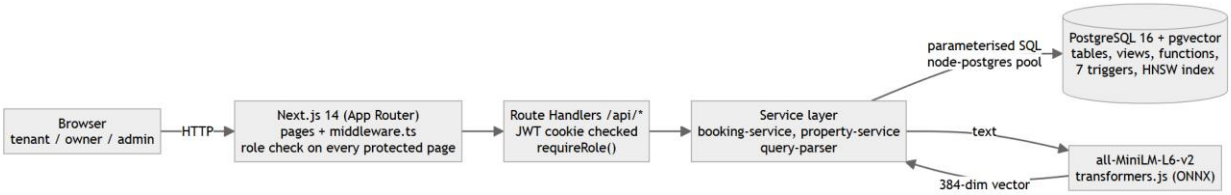


Fig. 9.1: System architecture

9.2 PROJECT STRUCTURE

```
propfind/
├── app/   # pages and API route handlers
│   ├── api/   # REST API
│   │   ├── auth/   # register, login, logout, me
│   │   │   ├── login/route.ts
│   │   │   ├── logout/route.ts
│   │   │   ├── me/route.ts
│   │   │   └── register/route.ts
│   │   ├── bookings/   # bookings, status, pay, history
│   │   │   ├── [id]/
│   │   │   │   ├── pay/route.ts
│   │   │   │   └── route.ts
│   │   │   └── route.ts
│   │   ├── owner/properties/route.ts
│   │   ├── properties/route.ts
│   │   └── search/route.ts
│   ├── dashboard/
│   │   ├── owner/   # owner portal and detail pages
│   │   │   ├── confirmed/page.tsx
│   │   │   ├── properties/page.tsx
│   │   │   ├── requests/page.tsx
│   │   │   ├── revenue/page.tsx
│   │   │   └── page.tsx
│   │   └── tenant/page.tsx
│   ├── login/page.tsx
│   ├── properties/new/page.tsx
│   ├── register/page.tsx
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx
├── components/   # shared React components
│   ├── Footer.tsx
│   ├── Navbar.tsx
│   ├── OwnerSubpageHeader.tsx
│   ├── PropertyCard.tsx
│   └── PropertyModal.tsx
├── db/   # schema, demo queries, tests
│   ├── queries.sql
│   ├── schema.sql
│   └── transactions_tests.sql
├── docs/   # documentation and report
│   ├── report/
│   │   ├── images/ (screenshots)
│   │   ├── PropFind_Project_Report.docx
│   │   └── PropFind_Project_Report.pdf
│   ├── ARCHITECTURE.md
```

```
│   ├── DATABASE_DESIGN.md
│   ├── DEV_LOGS.md
│   ├── HOSTED_DATABASE.md
│   ├── MEMBER_2_GUIDE.md
│   └── TEAM_DIVISION.md
├── lib/   # server-side libraries
│   ├── services/   # booking and property services
│   │   ├── booking-service.ts
│   │   └── property-service.ts
│   ├── auth.ts
│   ├── db.ts
│   ├── embeddings.ts
│   ├── jwt.ts
│   └── query-parser.ts
├── scripts/   # db:init, db:seed, seed data
│   ├── init-db.mjs
│   ├── seed-data.mjs
│   ├── seed.mjs
│   └── test-parser.mjs
├── .env.example
├── .gitignore
├── DBMS Level 3 Mini_Project Requirements draft.docx
├── README.md
├── docker-compose.yml
├── middleware.ts   # role-based route protection
├── next.config.mjs
├── package.json
├── postcss.config.mjs
├── tailwind.config.ts
└── tsconfig.json
```

9.3 AUTHENTICATION AND ROLE-BASED ACCESS

middleware.ts (role-based route protection)

```ts
/**
 * Role-based access control: runs before every protected page loads.
 * (API routes additionally re-check the role themselves via requireRole().)
 */
import { NextResponse, type NextRequest } from 'next/server';
import { verifyToken, COOKIE, type Role } from '@/lib/jwt';

const rules: { prefix: string; roles: Role[] }[] = [
  { prefix: '/dashboard/owner', roles: ['OWNER', 'ADMIN'] },
  { prefix: '/dashboard/tenant', roles: ['TENANT', 'ADMIN'] },
  { prefix: '/properties/new', roles: ['OWNER', 'ADMIN'] },
];

export async function middleware(req: NextRequest) {
  const rule = rules.find((r) => req.nextUrl.pathname.startsWith(r.prefix));
  if (!rule) return NextResponse.next();

  const session = await verifyToken(req.cookies.get(COOKIE)?.value);
  if (!session) {
    const url = new URL('/login', req.url);
    url.searchParams.set('next', req.nextUrl.pathname);
    return NextResponse.redirect(url);
  }
  if (!rule.roles.includes(session.role)) {
    return NextResponse.redirect(new URL('/', req.url));
  }
  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*', '/properties/new'],
};
```

Fig. 9.2: Role-based protection of pages (middleware.ts)

lib/jwt.ts + lib/auth.ts (JWT session with jose, httpOnly cookie)

```ts
export async function createToken(s: Session): Promise<string> {
  return new SignJWT({ name: s.name, role: s.role })
    .setProtectedHeader({ alg: 'HS256' })
    .setSubject(s.userId)
    .setIssuedAt()
    .setExpirationTime('7d')
    .sign(secret());
}

export async function requireRole(...roles: Role[]): Promise<Session | null> {
  const s = await getSession();
  if (!s || !roles.includes(s.role)) return null;
  return s;
}

export async function setSessionCookie(res: NextResponse, s: Session) {
  res.cookies.set(COOKIE, await createToken(s), {
    httpOnly: true,
    sameSite: 'lax',
    path: '/',
    maxAge: COOKIE_MAX_AGE,
    secure: process.env.COOKIE_SECURE === 'true', // enable when served over HTTPS
  });
}
```

Fig. 9.3: JWT session helpers and role check (lib/jwt.ts, lib/auth.ts)

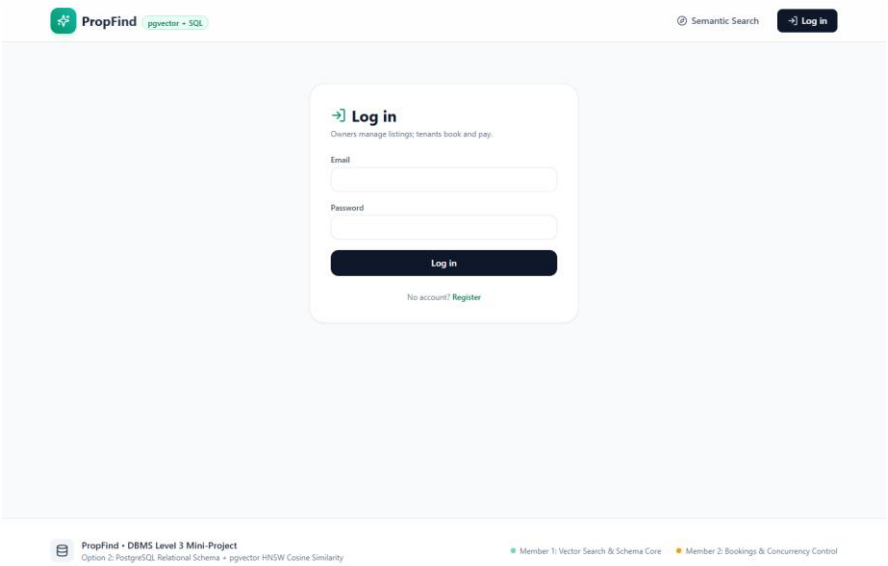


Fig. 9.4: Login page

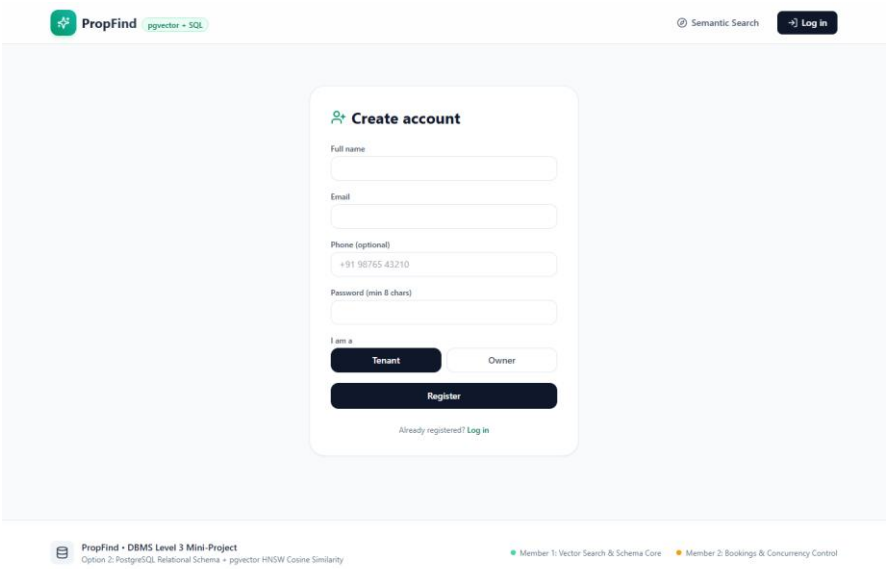


Fig. 9.5: Registration page (owner or tenant)

9.4 SEARCH PAGES

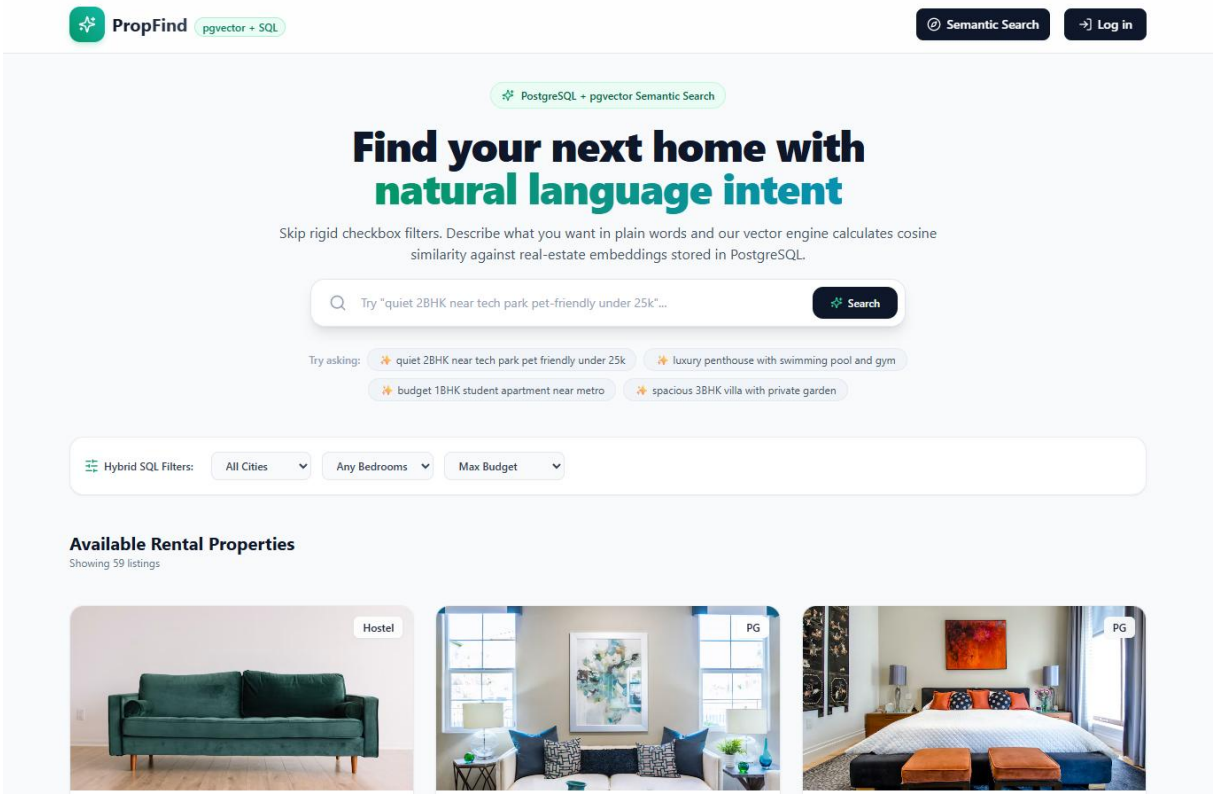


Fig. 9.6: Home page

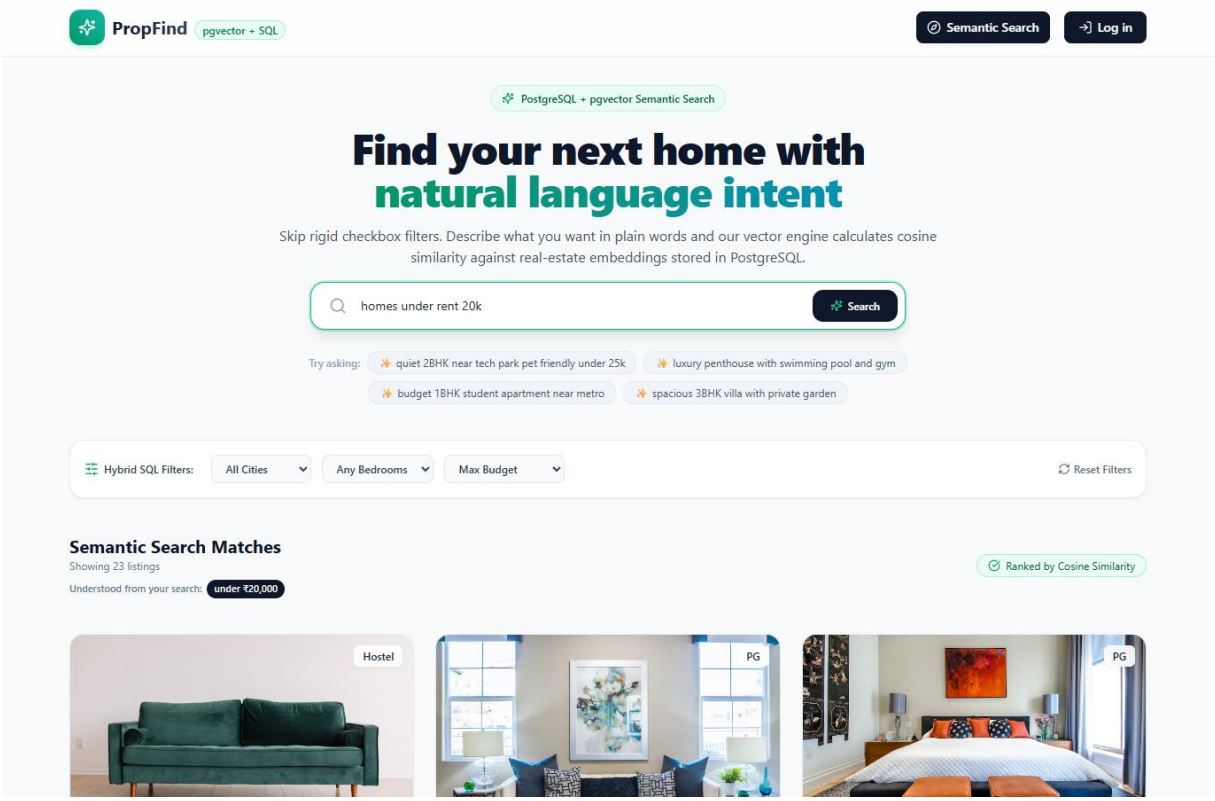


Fig. 9.7: Search "homes under rent 20k": 23 results, all under Rs. 20,000, with the understood constraint shown

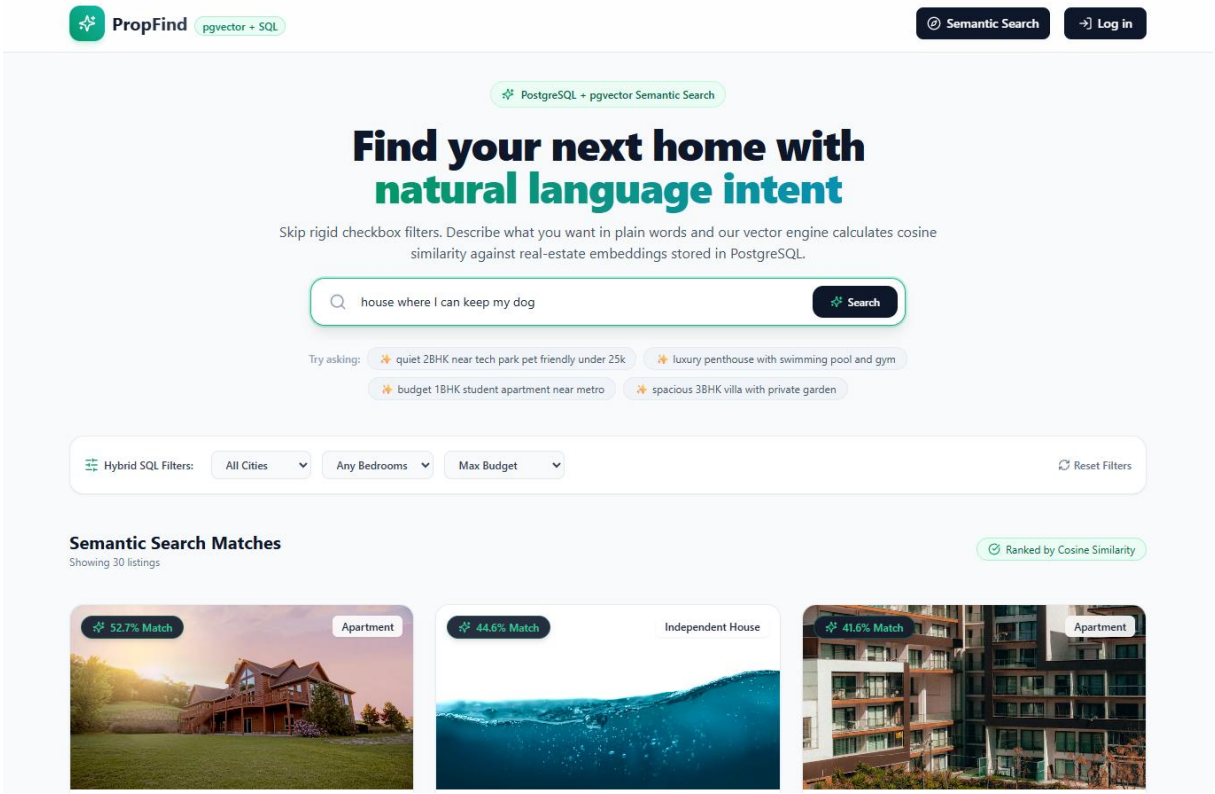


Fig. 9.8: Search "house where I can keep my dog": semantic ranking puts pet-friendly homes first

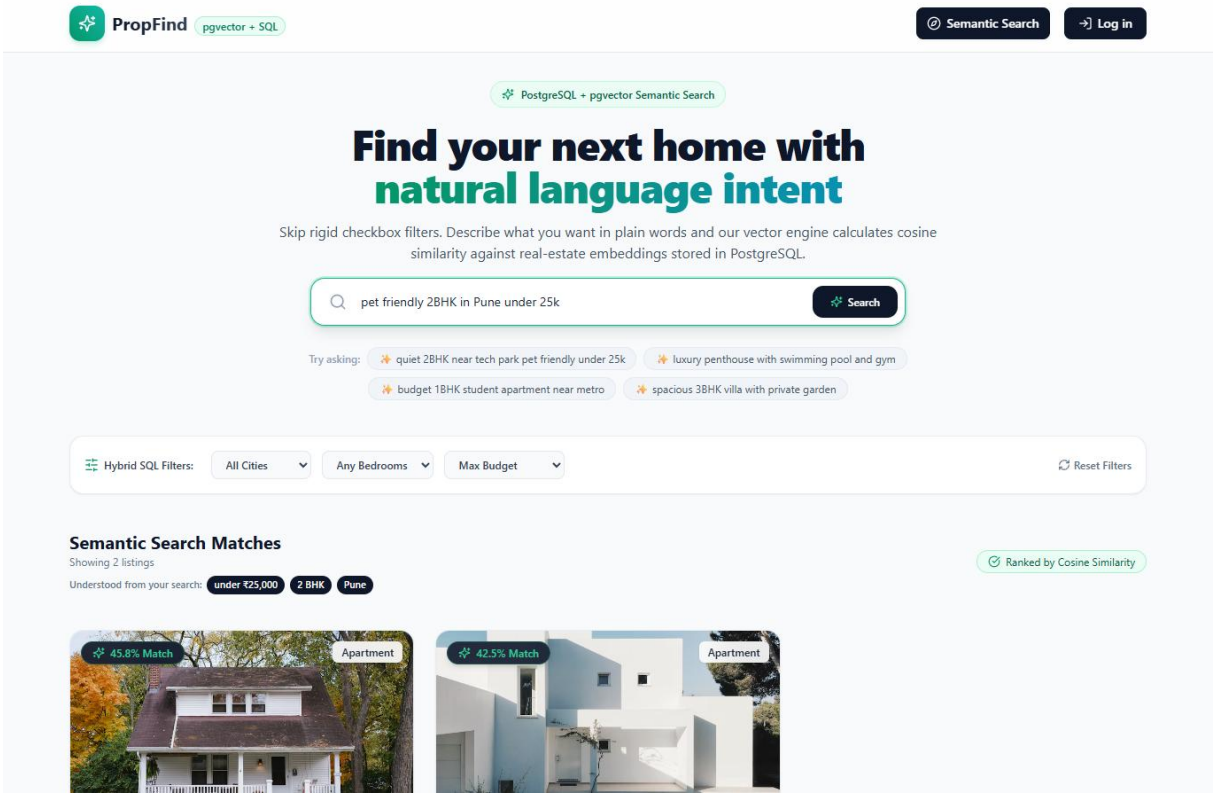


Fig. 9.9: Search "pet friendly 2BHK in Pune under 25k": bedrooms, city and budget parsed into SQL filters

lib/query-parser.ts (budget, bedrooms and city extracted from the sentence)

```ts
function toRupees(raw: string, unit?: string): number {
  const n = parseFloat(raw.replace(/,/g, ''));
  const u = (unit || '').toLowerCase();
  if (u === 'k' || u === 'thousand') return Math.round(n * 1_000);
  if (u === 'l' || u === 'lac' || u === 'lacs' || u === 'lakh' || u === 'lakhs') return Math.round(n * 100_000);
  return Math.round(n);
}

const fmt = (n: number) => '₹' + n.toLocaleString('en-IN');

export function parseSearchQuery(input: string, knownCities: string[] = []): ParsedQuery {
  let text = ` ${input} `;
  const applied: string[] = [];
  const result: ParsedQuery = { text: '', isGeneric: false, applied };

  // Budget ceiling
  let m = text.match(MAX_RE);
  if (m) {
    const v = toRupees(m[1], m[2]);
    // a bare small number ("under 3") is more likely bedrooms than rupees: ignore it
    if (v >= 1000) {
      result.maxPrice = v;
      applied.push(`under ${fmt(v)}`);
      text = text.replace(MAX_RE, ' ');
    }
  }

  // Budget floor
  m = text.match(MIN_RE);
  if (m) {
    const v = toRupees(m[1], m[2]);
    if (v >= 1000) {
      result.minPrice = v;
      applied.push(`above ${fmt(v)}`);
      text = text.replace(MIN_RE, ' ');
    }
  }

  // Loose phrasing ("20k budget", "rent 15000") sets a ceiling if none was found yet
  if (result.maxPrice === undefined && result.minPrice === undefined) {
    m = text.match(LOOSE_RE);
    if (m) {
      const raw = m[1] ?? m[3];
      const unit = m[1] ? m[2] : m[4];
      const v = toRupees(raw, unit);
      if (v >= 1000) {
        result.maxPrice = v;
        applied.push(`under ${fmt(v)}`);
        text = text.replace(LOOSE_RE, ' ');
      }
    }
  }
```

Fig. 9.10: Extracting budget, bedrooms and city from the sentence (lib/query-parser.ts)

lib/services/property-service.ts (hybrid search: parsed filters + vector ranking)

```ts
  // Semantic path: embed only the descriptive part of the sentence
  const queryEmbedding = await getEmbedding(parsed.text);
  const vectorStr = toVectorLiteral(queryEmbedding);

  const sql = `
    SELECT
      id, title, description, property_type, price_per_month, bedrooms, bathrooms,
      area_sqft, address, city, status, amenities, image_url, owner_name, avg_rating,
      ROUND((similarity_score * 100)::numeric, 1)::float AS similarity_score
    FROM fn_search_properties_semantic($1::text::vector, 0.20, $2, $3, $4, $5)
    WHERE ($6::numeric IS NULL OR price_per_month >= $6)
  `;
  const result = await query<Property>(sql, [
    vectorStr,
    limit ?? 30,
    city ?? null,
    maxPrice ?? null,
    bedrooms ?? null,
    minPrice ?? null,
  ]);
  return { properties: result.rows.map(mapRow), applied, semanticText: parsed.text };
}
```

Fig. 9.11: Hybrid search service (lib/services/property-service.ts)

lib/embeddings.ts (real all-MiniLM-L6-v2 embedding, mean pooling + normalisation)

```ts
/** all-MiniLM-L6-v2 embedding or semantic vectorizer fallback */
export async function generateLocalEmbedding(text: string): Promise<number[]> {
  try {
    const extractor = await getExtractor();
    if (extractor) {
      const out = await extractor(text, { pooling: 'mean', normalize: true });
      const vec = Array.from(out.data as Float32Array);
      if (vec.length === EMBEDDING_DIMENSION) {
        return vec;
      }
    }
  } catch (err) {
    useOfflineFallback = true;
  }
  return generateSemanticHashEmbedding(text);
}
```

Fig. 9.12: Generating a 384-dimension embedding (lib/embeddings.ts)

9.5 TENANT SCREENS

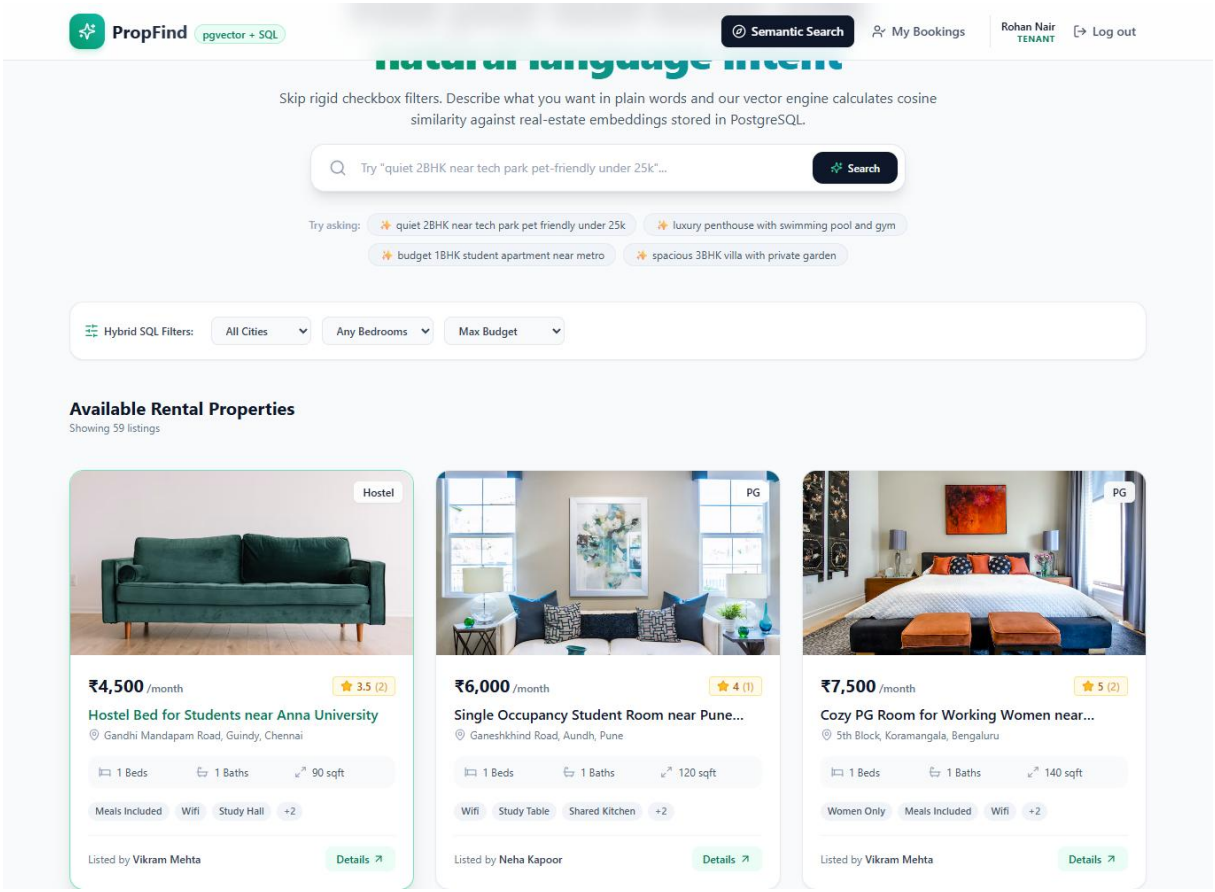


Fig. 9.13: Listing details

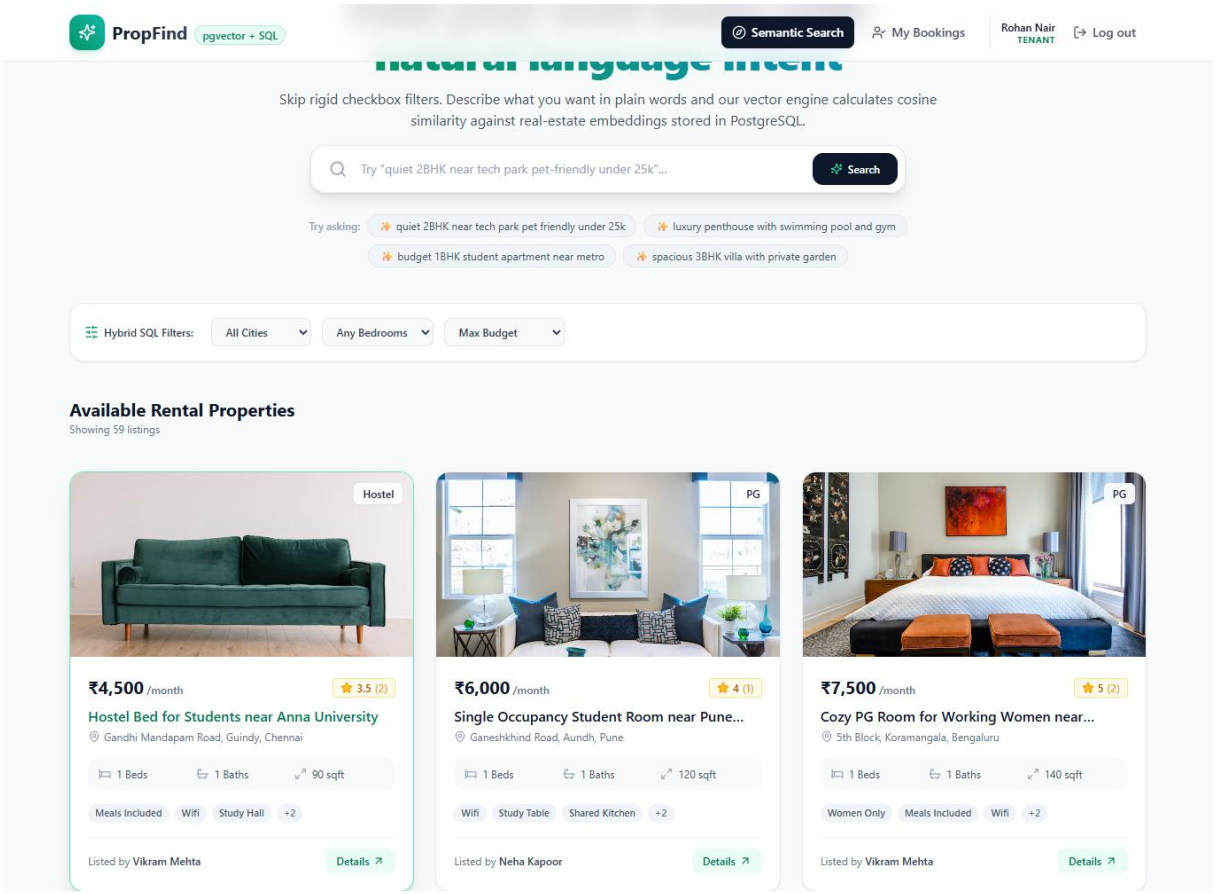


Fig. 9.14: Booking request form (runs as a PostgreSQL transaction)

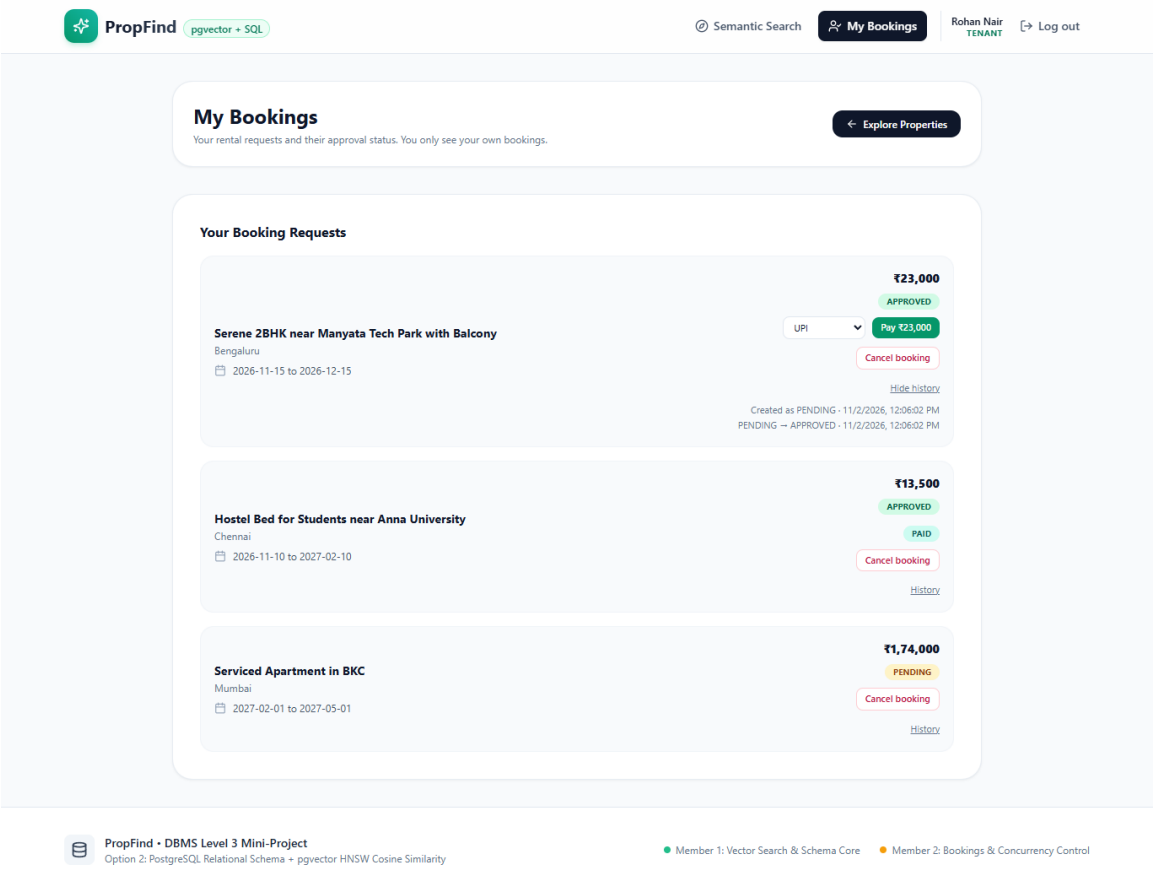


Fig. 9.15: Tenant dashboard: own bookings only, Pay, Cancel and status history from the audit table

9.6 OWNER SCREENS

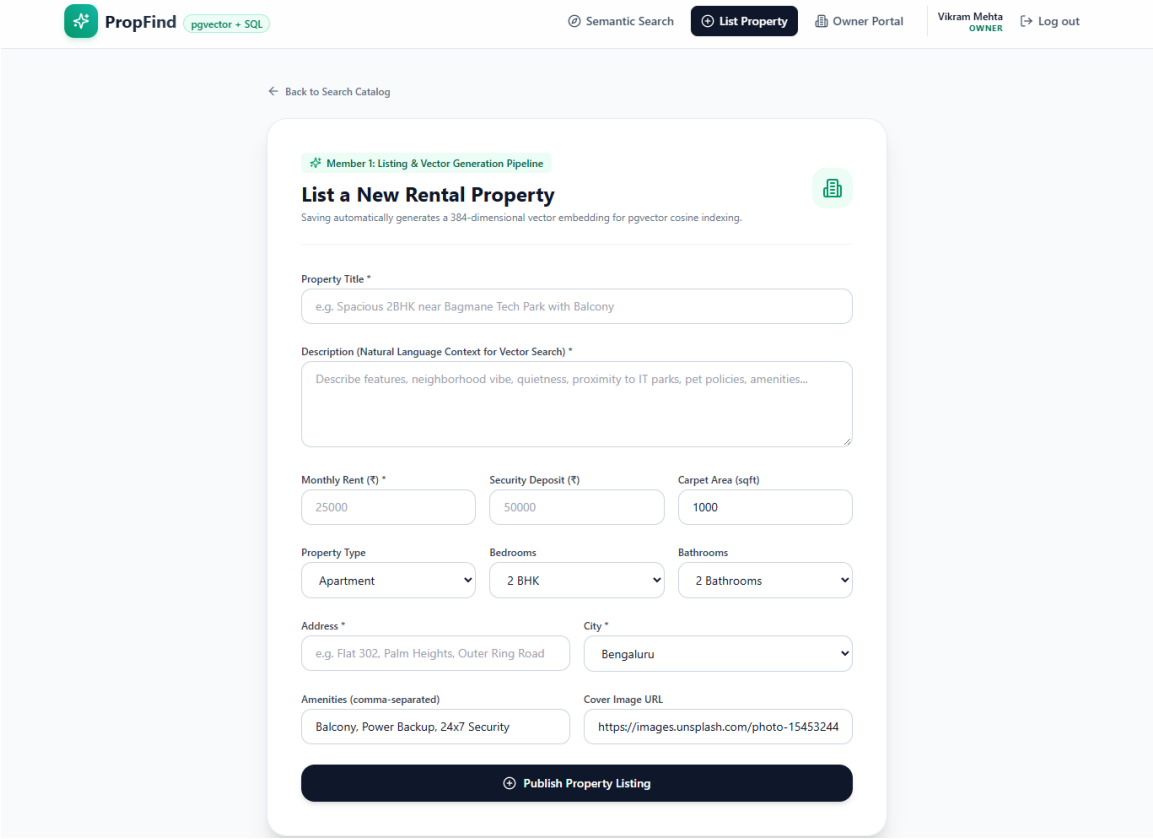


Fig. 9.16: Listing form (the embedding is created in the same transaction)

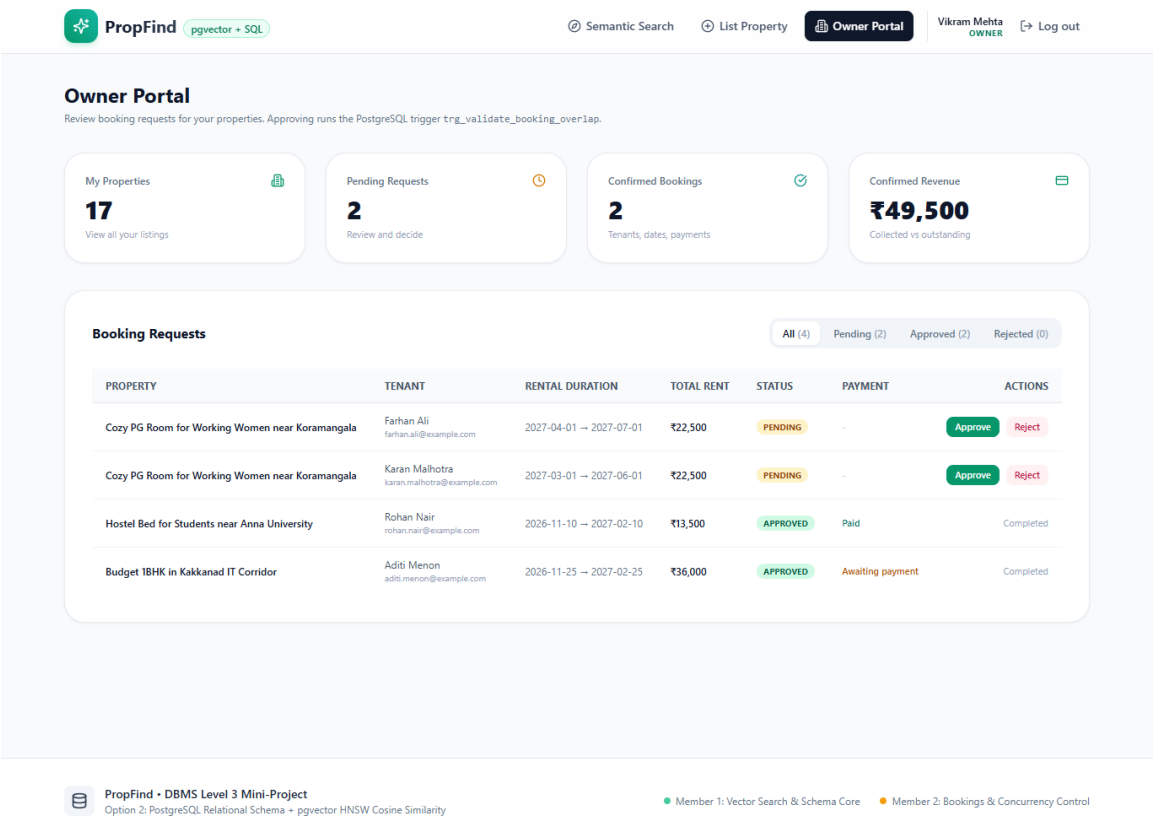


Fig. 9.17: Owner Portal: four clickable cards (from view_owner_analytics) and booking tabs

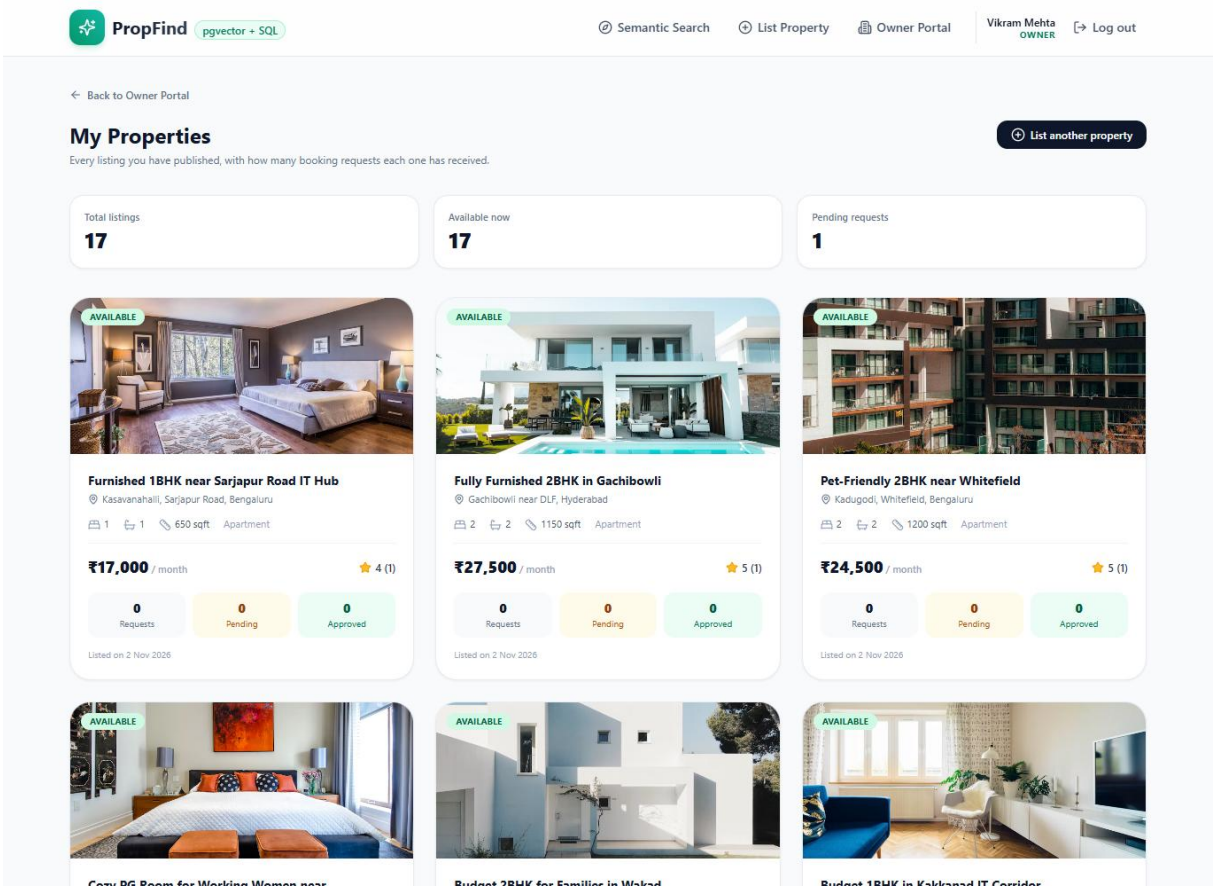


Fig. 9.18: My Properties

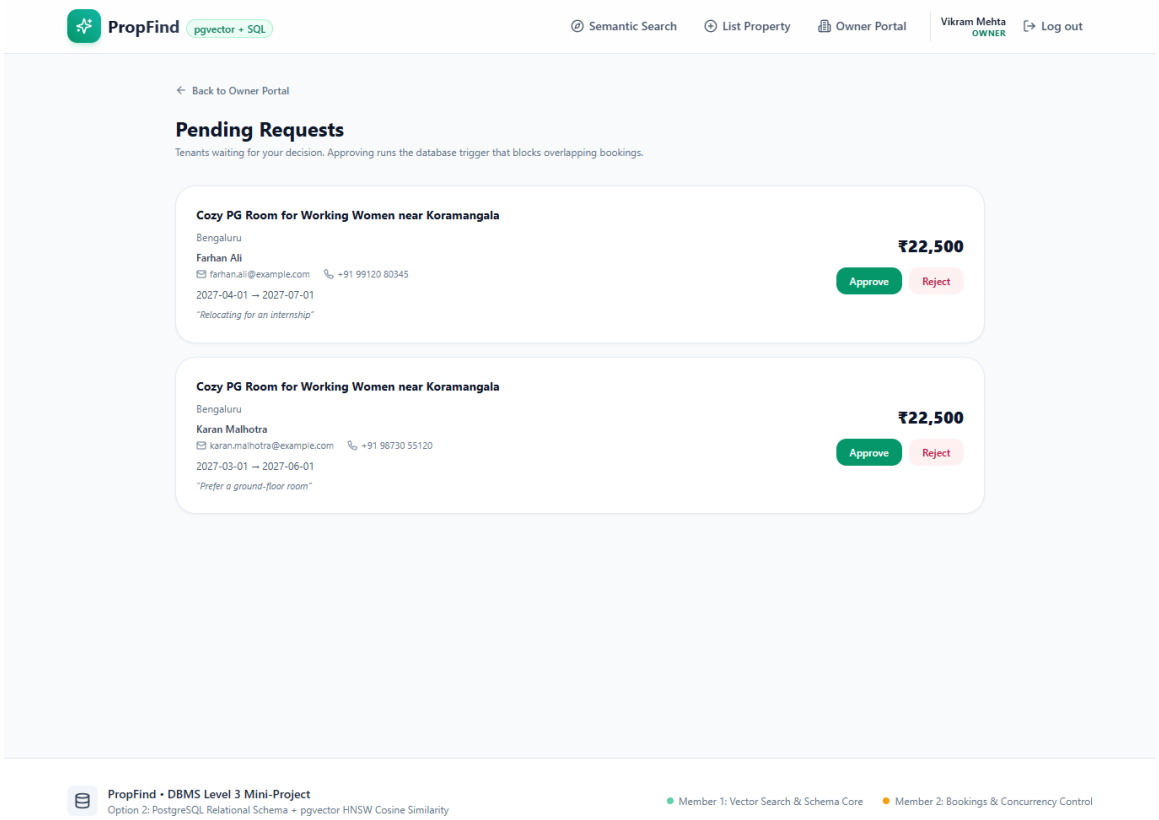


Fig. 9.19: Pending requests

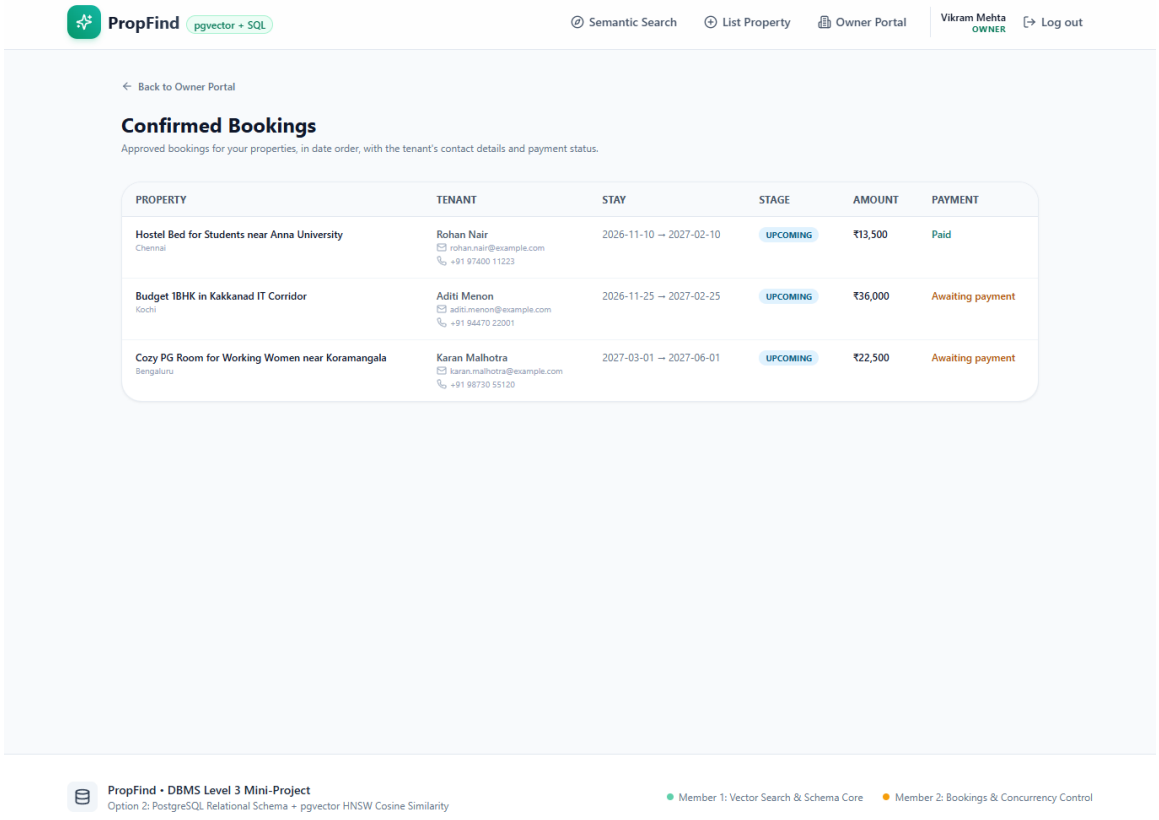


Fig. 9.20: Confirmed bookings

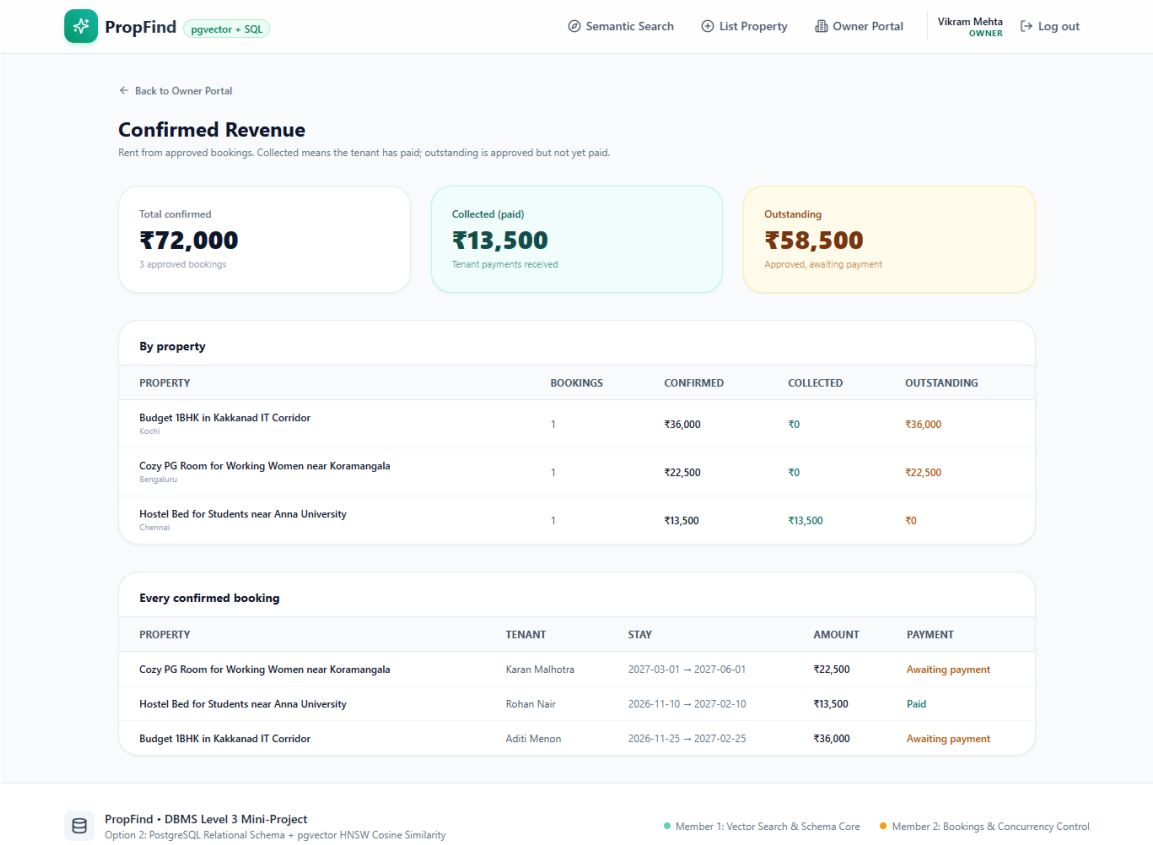


Fig. 9.21: Confirmed revenue: collected and outstanding, per property

9.7 TESTING RESULTS

Table 9.1: Tests performed

Test	How to run	Result
Database integrity and transactions (23 checks)	psql -f db/transactions_tests.sql	23 passed, 0 failed
Labelled SQL demonstrations	psql -f db/queries.sql	Exit code 0; data unchanged afterwards
Natural-language parser (12 cases)	npm run test:parser	12 of 12 passed
Production build	npm run build	Passes (13 routes)
End-to-end flow with scripted requests	register, login, search, book, approve, pay, double-booking rejection, role redirects	All behaved as expected
Access control	Anonymous, tenant and owner calls to protected pages and APIs	401 / 403 or redirect as required

9.8 CONCLUSION AND FUTURE ENHANCEMENTS

PropFind shows how one PostgreSQL database can serve both paradigms required by the project: a relational core whose integrity is guaranteed by constraints, transactions and triggers, and a vector layer that matches meaning instead of keywords. The most important design decision was to keep hard numeric

constraints in SQL and to use vectors only for what vectors do well. The application is backed by SQL scripts and tests that can be re-run from the repository.

Future enhancements

- A real payment gateway and e-mail notifications.
- An admin screen, property editing and deletion, image upload.
- A dedicated GIN index for full-text search, an availability calendar, lease extensions and saved listings.
- Automated browser tests, rate limiting and CSRF tokens for production use.
- A hosted database (the configuration option and guide are already in the repository).

REFERENCES

[1] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation". https://www.postgresql.org/docs/16/

[2] pgvector: open-source vector similarity search for PostgreSQL. https://github.com/pgvector/pgvector

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", EMNLP 2019. https://arxiv.org/abs/1908.10084; model card: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[4] Y. Malkov and D. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs", IEEE TPAMI, 2020. https://arxiv.org/abs/1603.09320

[5] Hugging Face, "Transformers.js documentation". https://huggingface.co/docs/transformers.js

[6] Vercel, "Next.js Documentation". https://nextjs.org/docs

[7] node-postgres documentation. https://node-postgres.com

[8] R. Elmasri and S. Navathe, "Fundamentals of Database Systems", Pearson (ER to relational mapping algorithm).

[9] jose (JSON Web Token library) https://github.com/panva/jose and bcryptjs https://github.com/dcodeIO/bcrypt.js

GitHub Link: https://github.com/jishnu-sivakumar/DBMS_mini

APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Table A.1: Definitions and abbreviations

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability: the guarantees of a database transaction
API	Application Programming Interface (here: the /api route handlers)
bcrypt	Password hashing function that stores a salted, slow hash instead of the password
CHECK	SQL constraint that rejects rows that do not satisfy a condition
Cosine distance	Measure of the angle between two vectors; 0 means identical direction. pgvector operator <=>
DDL / DML	Data Definition Language (CREATE, ALTER, DROP) / Data Manipulation Language (INSERT, UPDATE, DELETE, SELECT)
Embedding	A list of numbers (here 384) that represents the meaning of a text
ENUM	Column type restricted to a fixed list of values
ER	Entity-Relationship (diagram)
FK / PK	Foreign key / primary key
HNSW	Hierarchical Navigable Small World: an index for approximate nearest-neighbour search
JWT	JSON Web Token: a signed token used here for login sessions
pgvector	PostgreSQL extension that adds a vector type, distance operators and vector indexes
RBAC	Role-based access control
SRN	Student registration number
Trigger	Function that the database runs automatically before or after INSERT, UPDATE or DELETE
UUID	Universally unique identifier used for primary keys

APPENDIX B: DATA DICTIONARY

Generated from information_schema on the running database (PostgreSQL 16.15, pgvector 0.8.7).

Table B.1: Table users

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
full_name	character varying(120)	NOT NULL	
email	character varying(160)	NOT NULL	
password_hash	character varying(255)	NOT NULL	
role	user_role	NOT NULL	'TENANT'::user_role
phone	character varying(20)	NULL	
avatar_url	text	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP
updated_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

Table B.2: Table properties

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
owner_id	uuid	NOT NULL	
title	character varying(200)	NOT NULL	
description	text	NOT NULL	
property_type	character varying(50)	NOT NULL	'Apartment'::character varying
price_per_month	numeric(10,2)	NOT NULL	
security_deposit	numeric(10,2)	NOT NULL	0.00
bedrooms	integer	NOT NULL	
bathrooms	integer	NOT NULL	
area_sqft	integer	NOT NULL	
address	text	NOT NULL	
city	character varying(80)	NOT NULL	
state	character varying(80)	NOT NULL	
pincode	character varying(20)	NOT NULL	
status	property_status	NOT NULL	'AVAILABLE'::property_status
amenities	text[]	NULL	'{}'::text[]
image_url	text	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP
updated_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

Table B.3: Table property_embeddings

Column	Type	Null	Default
property_id	uuid	NOT NULL	

Column	Type	Null	Default
embedding	vector	NOT NULL	
model_name	character varying(100)	NOT NULL	'all-MiniLM-L6-v2'::character varying
indexed_content	text	NOT NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP
updated_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

Table B.4: Table bookings

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
property_id	uuid	NOT NULL	
tenant_id	uuid	NOT NULL	
start_date	date	NOT NULL	
end_date	date	NOT NULL	
total_amount	numeric(10,2)	NOT NULL	
status	booking_status	NOT NULL	'PENDING'::booking_status
special_requests	text	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP
updated_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

Table B.5: Table reviews

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
property_id	uuid	NOT NULL	
tenant_id	uuid	NOT NULL	
rating	integer	NOT NULL	
comment	text	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

Table B.6: Table payments

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
booking_id	uuid	NOT NULL	
amount	numeric(10,2)	NOT NULL	
method	payment_method	NOT NULL	
status	payment_status	NOT NULL	'PAID'::payment_status
transaction_ref	character varying(40)	NOT NULL	('TXN-'::text || upper(substr(md5(((ra
paid_at	timestamp with time zone	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

Table B.7: Table booking_audit

Column	Type	Null	Default
id	bigint	NOT NULL	nextval('booking_audit_id_seq'::regcla
booking_id	uuid	NOT NULL	
old_status	booking_status	NULL	
new_status	booking_status	NOT NULL	
changed_by	uuid	NULL	
changed_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

APPENDIX C: EVALUATION RUBRICS - COMPLETION STATUS

Status key: Done = implemented and demonstrated; Partial = implemented with a stated limitation; Prepared = ready but not deployed.

Table C.1: Completion status against the project brief

Requirement	Status	Evidence / note
Two-member team with defined responsibilities; both database and application work	Done	Section 1.7; both members have SQL and application commits
Option 2: SQL (PostgreSQL) + vector database (pgvector)	Done	Chapters 3 to 8
ER diagram converted to relational schema; entities, attributes, keys, relationships, constraints	Done	Fig. 3.1, Chapter 4, Appendix B
Vector data model documented (embeddings, metadata, index, retrieval)	Done	Section 3.5
DDL: CREATE, ALTER, DROP, keys, constraints, indexes, views	Done	Chapter 5
DML: INSERT, UPDATE, DELETE, SELECT	Done	Chapters 6 and 7
JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	Done	Sections 7.1 to 7.7 (30+ labelled queries in db/queries.sql)
At least four advanced concepts	Done	Transactions, views, functions, 7 triggers, indexing, vector similarity search
Full-text search	Partial	Shown as an ad-hoc tsvector query (Section 7.9); no dedicated GIN index or UI
Explanation of embeddings (what, model, storage, metric, retrieval)	Done	Section 3.5
Functional Next.js interface with forms, tables, search, filters, dashboards	Done	Chapter 9
Backend / API layer between Next.js and the database	Done	Section 9.1
No hard-coded credentials; environment variables	Done	.env and .env.example; the app does not start without DATABASE_URL. The seed script has a documented demo-password fallback for demo accounts only
Authentication and at least 2 roles	Done	Owner and tenant with checks in middleware and in every API route
Admin features	Partial	An admin account is seeded, but there is no dedicated admin screen
Validation at the database level	Done	ENUM, CHECK, UNIQUE, FK and triggers; 22 negative tests
Security basics	Done	bcrypt, httpOnly JWT cookie, parameterised SQL. Not done: rate limiting and CSRF tokens (SameSite=Lax only)
Documentation (problem, objectives, scope, ER, schema, DB choice, dictionary, queries, architecture, screenshots, tests, conclusion)	Done	This report, README and docs/
Git/GitHub with individual contributions and README with setup	Done	Pull requests from feature branches; 55 commits (Kanishq) and 2 commits (Jishnu) excluding merges

Requirement	Status	Evidence / note
Testing results	Partial	SQL 23 of 23, parser 12 of 12, scripted API flows; no automated browser tests
Originality; beyond CRUD	Done	Semantic and hybrid search, concurrency control, trigger-enforced state machine, audit log, refunds
Hosted database (optional)	Prepared	DATABASE_SSL option and docs/HOSTED_DATABASE.md; the demo database runs locally in Docker